CS11212 **- Spring** 2023

Data Structures & Introduction to **Algorithms**

Data Structures

Stacks & Queues

Ibrahim Albluwi

Stacks (a new ADT)

Stack

```
void push(const T& val)
    T pop()
    T top() const
```

add `val` to the collection of elements

remove the most recently added element

return the most recently added element

Stacks (a new ADT)

Stack

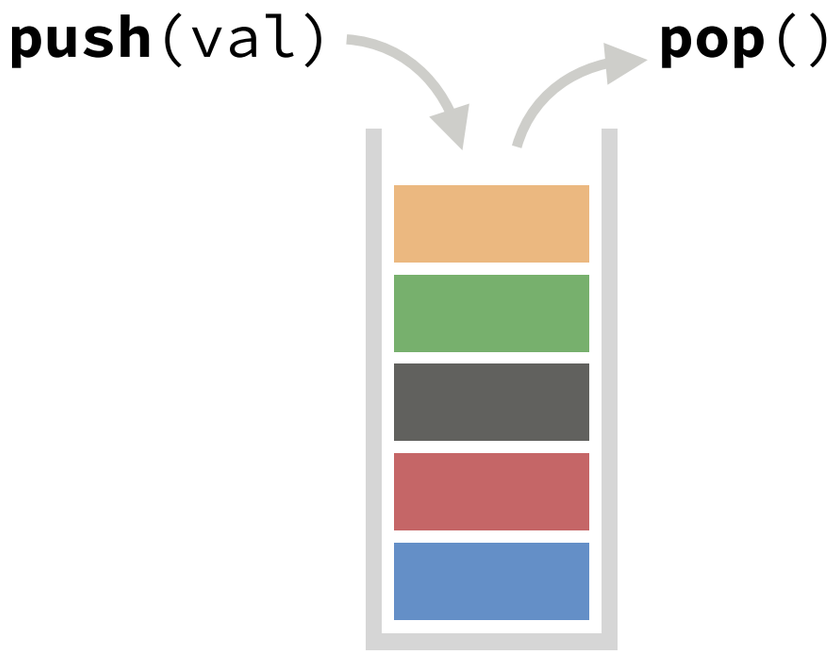
```
void push(const T& val)
    T pop()
    T top() const
```

add `val` to the collection of elements

remove the most recently added element

return the most recently added element

! Elements cannot be accessed or modified in any other way



LIFO: Last In First Out

~~**FILO**: First In Last Out~~

Stacks (a new ADT)

Stack

```
void push(const T& val)
    T pop()
    T top() const
```

add `val` to the collection of elements

remove the most recently added element

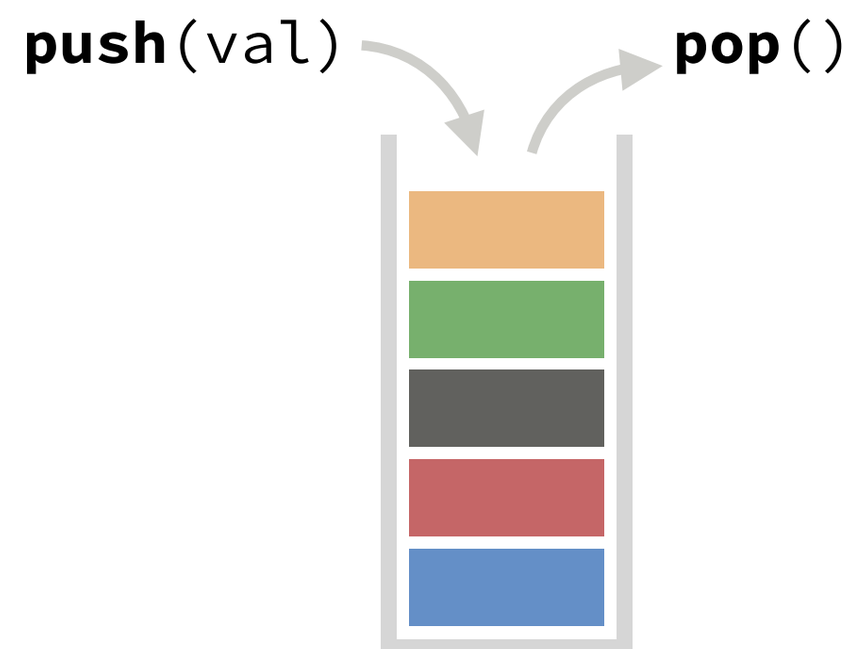
return the most recently added element

! Elements cannot be accessed or modified in any other way

مكـــدوس

Eat this first

Inaccessible until the makdous above is eaten.



LIFO: Last In First Out

~~**FILO**: First In Last Out~~

Exercise

What does the following code do?

```
void easy_to_trace(int a[], int size)

    Stack<int> s;
    for (int i = 0; i < size; i++)
        s.push(a[i]);

    for (int i = 0; i < size; i++)
        cout << s.pop() << endl;
```

Exercise

What does the following code do?

```
void easy_to_trace(int a[], int size)

    Stack<int> s;
    for (int i = 0; i < size; i++)
        s.push(a[i]);

    for (int i = 0; i < size; i++)
        cout << s.pop() << endl;
```

Answer. Prints the array elements in reverse order.

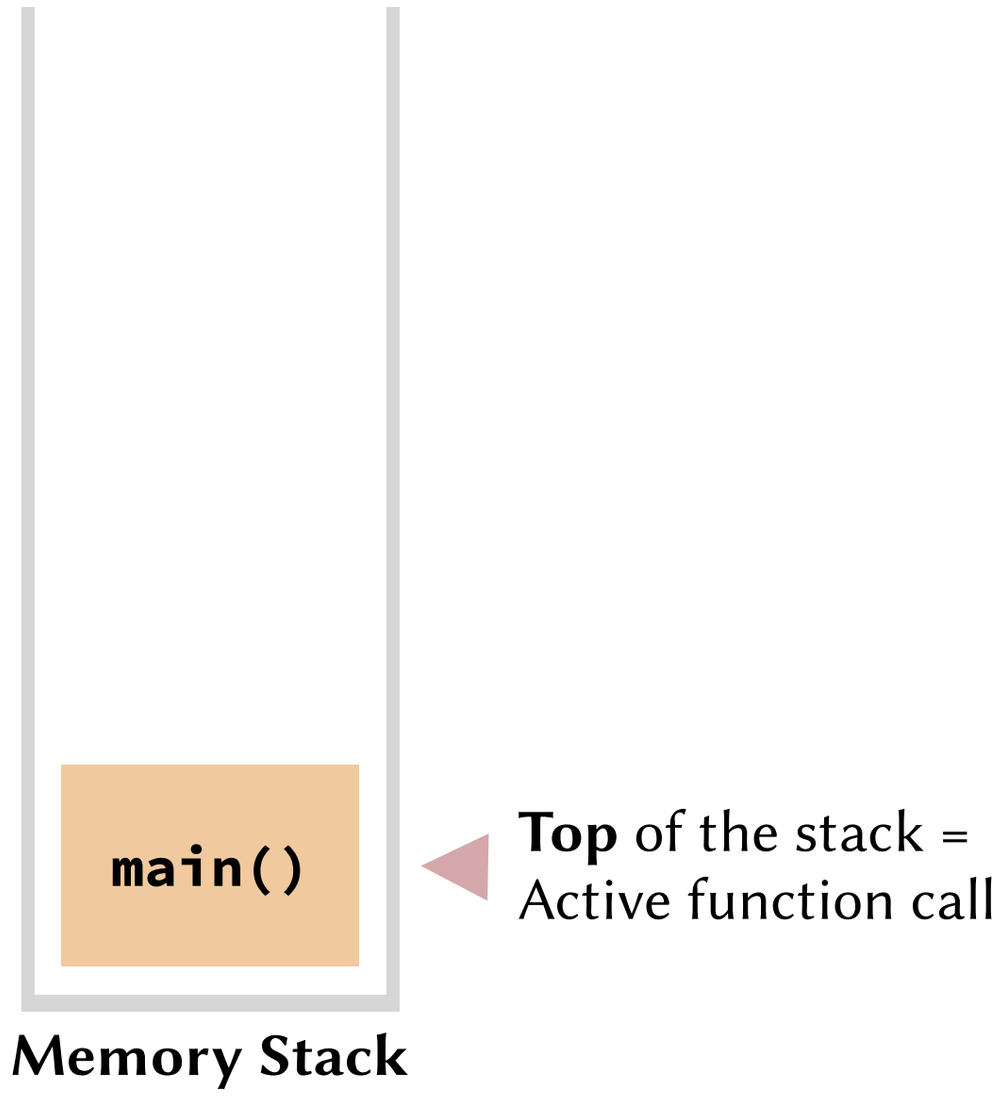
Application # 1

Stacks are used to ***reverse*** the order of a sequence.

Stacks (Applications)

1. Reversing sequences.
2. **Managing function calls.**

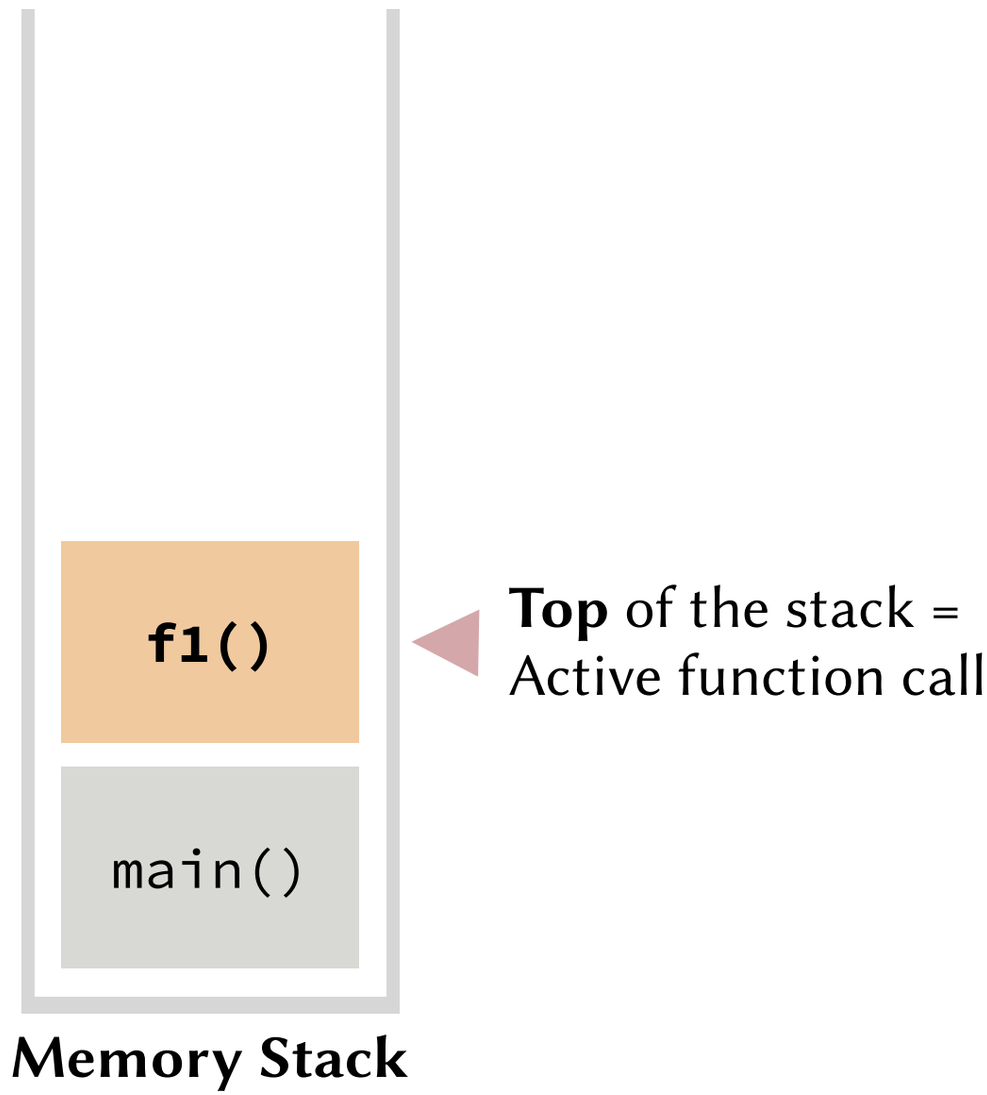
```
void f2() {
    ...
}

void f1() {
    ...
    f2();
    ...
}

int main() {
    ...
    f1();
    ...
}
```



Stacks (Applications)

1. Reversing sequences.
2. **Managing function calls.**

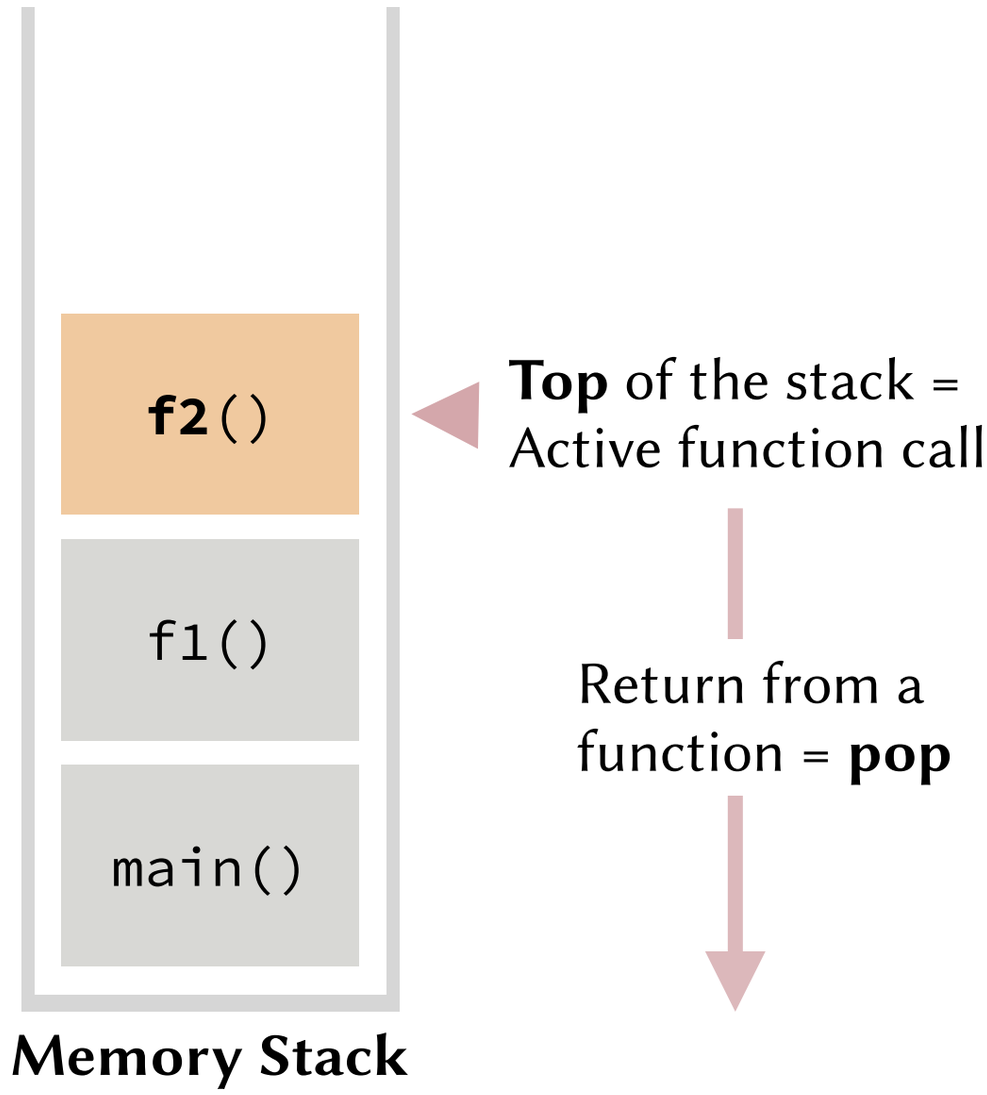
```
void f2() {
    ...
}

void f1() {
    ...
    f2();
    ...
}

int main() {
    ...
    f1();
    ...
}
```



Stacks (Applications)

1. Reversing sequences.
2. **Managing function calls.**

```
void f2() {
    ...
}

void f1() {
    ...
    f2();
    ...
}

int main() {
    ...
    f1();
    ...
}
```



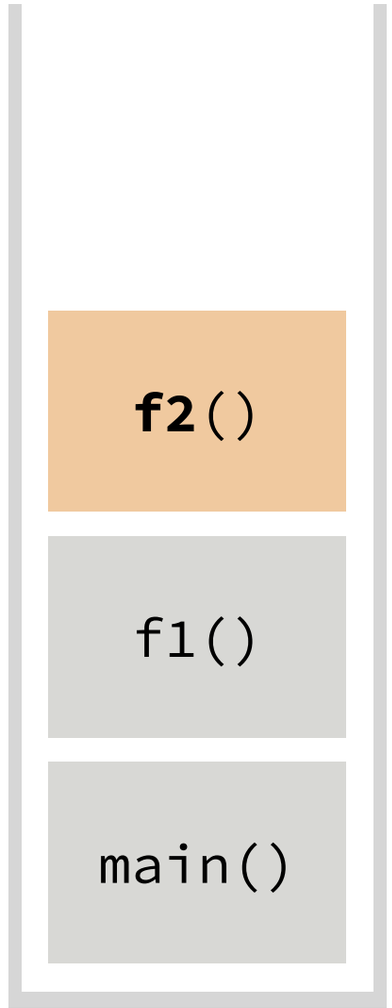
Stacks (Applications)

1. Reversing sequences.
2. **Managing function calls.**

Since **recursion** uses the memory stack, recursive algorithms typically can be converted to iterative algorithms using an explicit stack.

Stay tuned for examples when we cover the binary search tree data structure.

```
void f2() {
    ...
}

void f1() {
    ...
    f2();
    ...
}

int main() {
    ...
    f1();
    ...
}
```



Memory Stack

Top of the stack = Active function call

Return from a function = **pop**

Stacks (Applications)

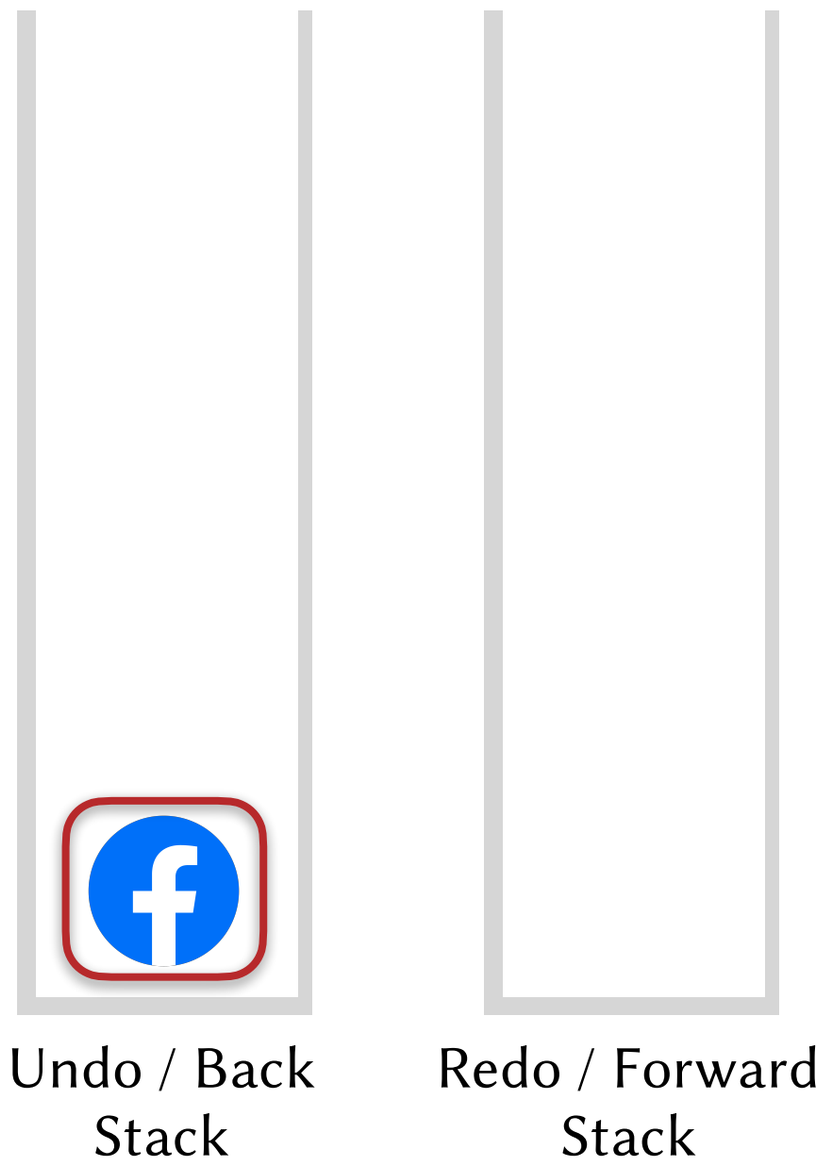
1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

Stacks (Applications)

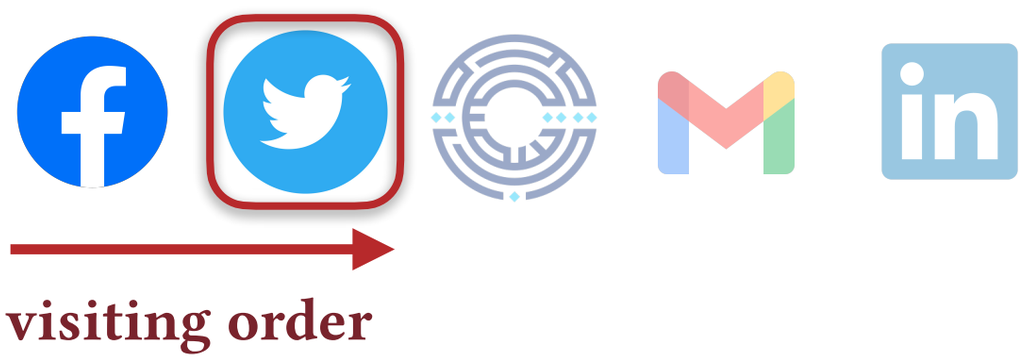
1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

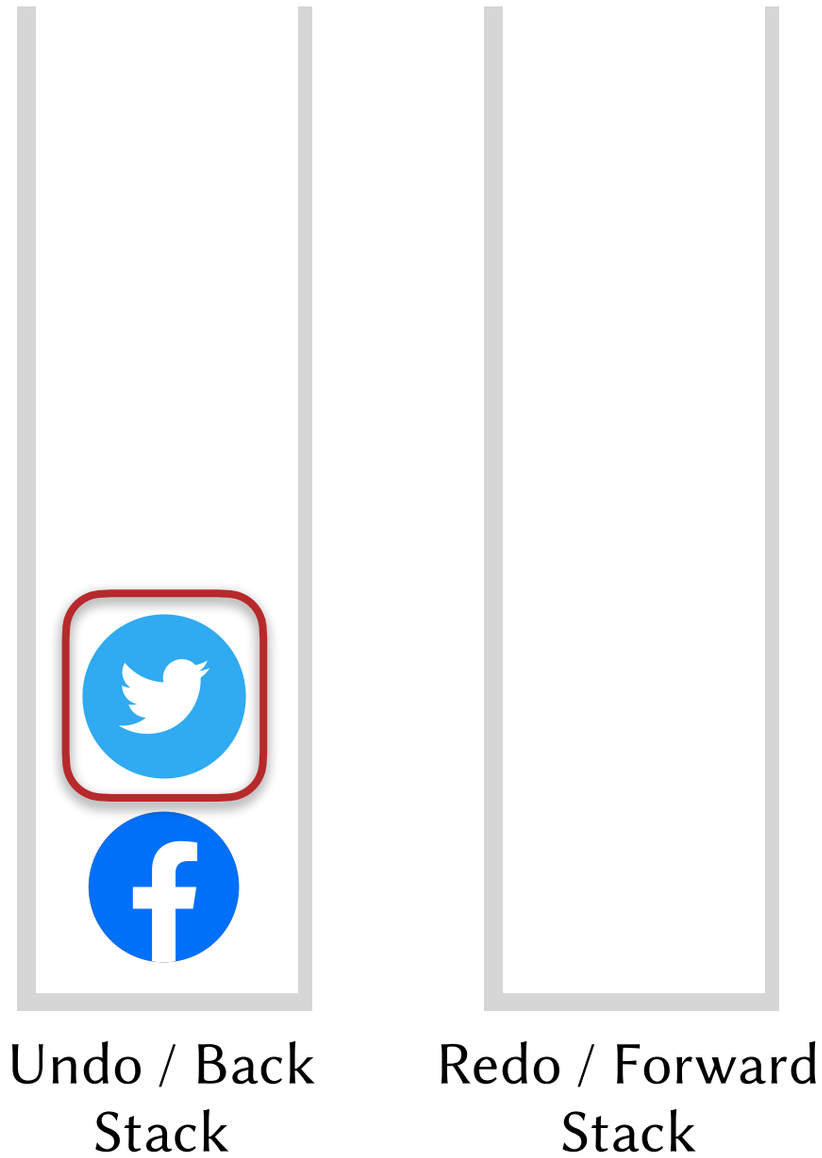



```
visit(page)

undo.push(page)
show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

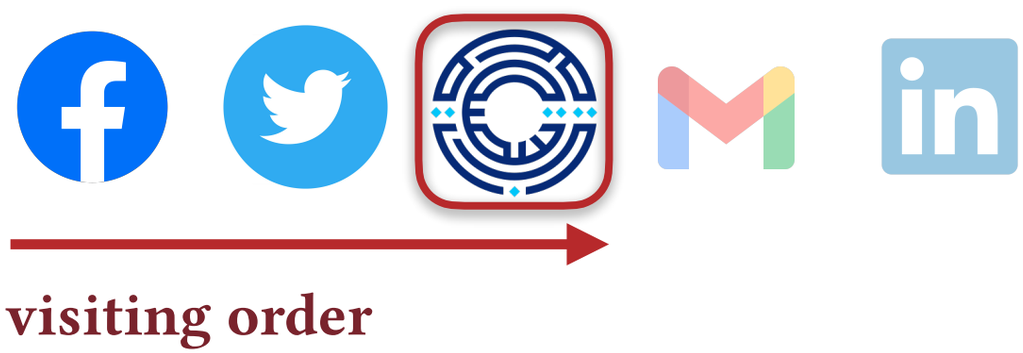


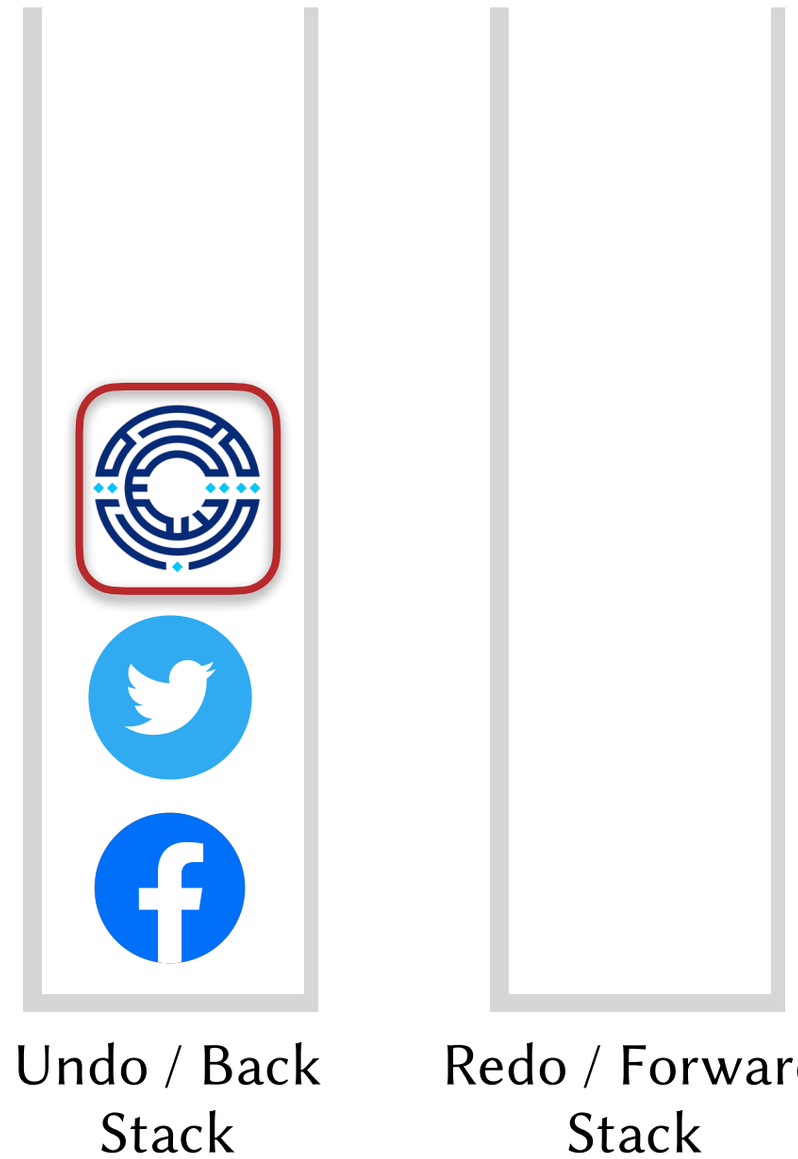



```
visit(page)

undo.push(page)
show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

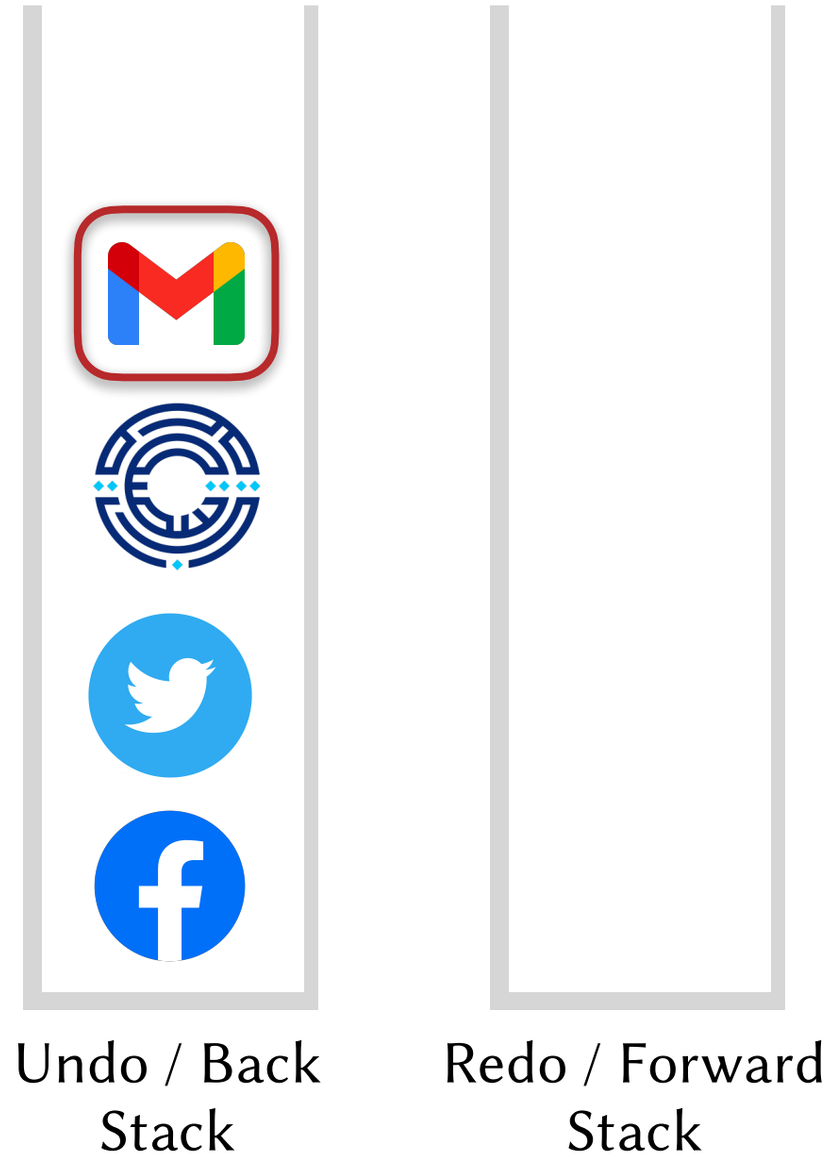





```
visit(page)

undo.push(page)
show(undo.top())
```

Stacks (Applications)

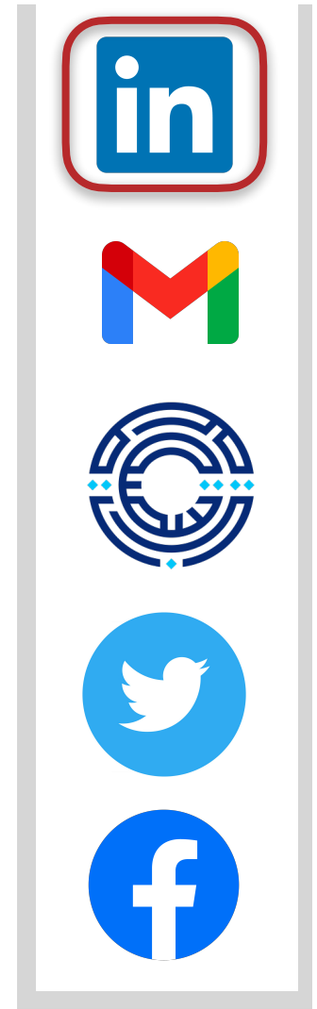
1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser




```
visit(page)

undo.push(page)
show(undo.top())
```

Stacks (Applications)

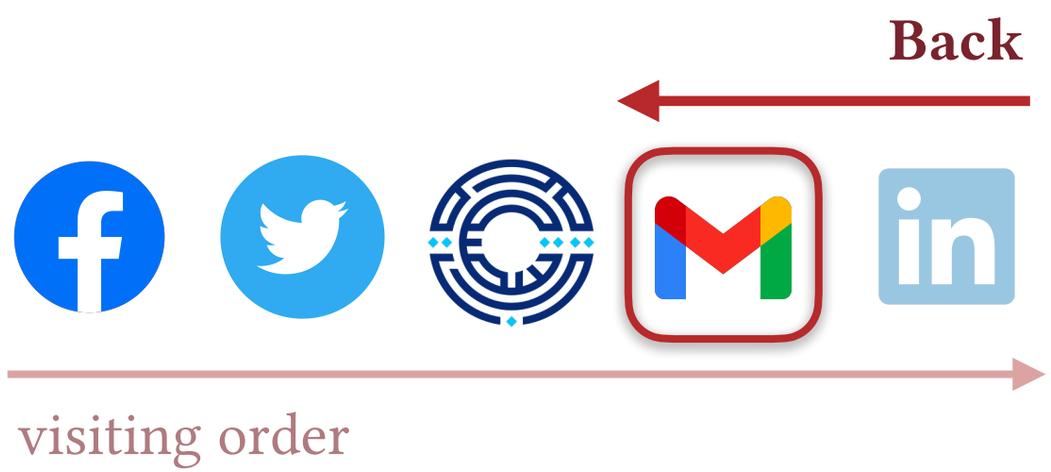
1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

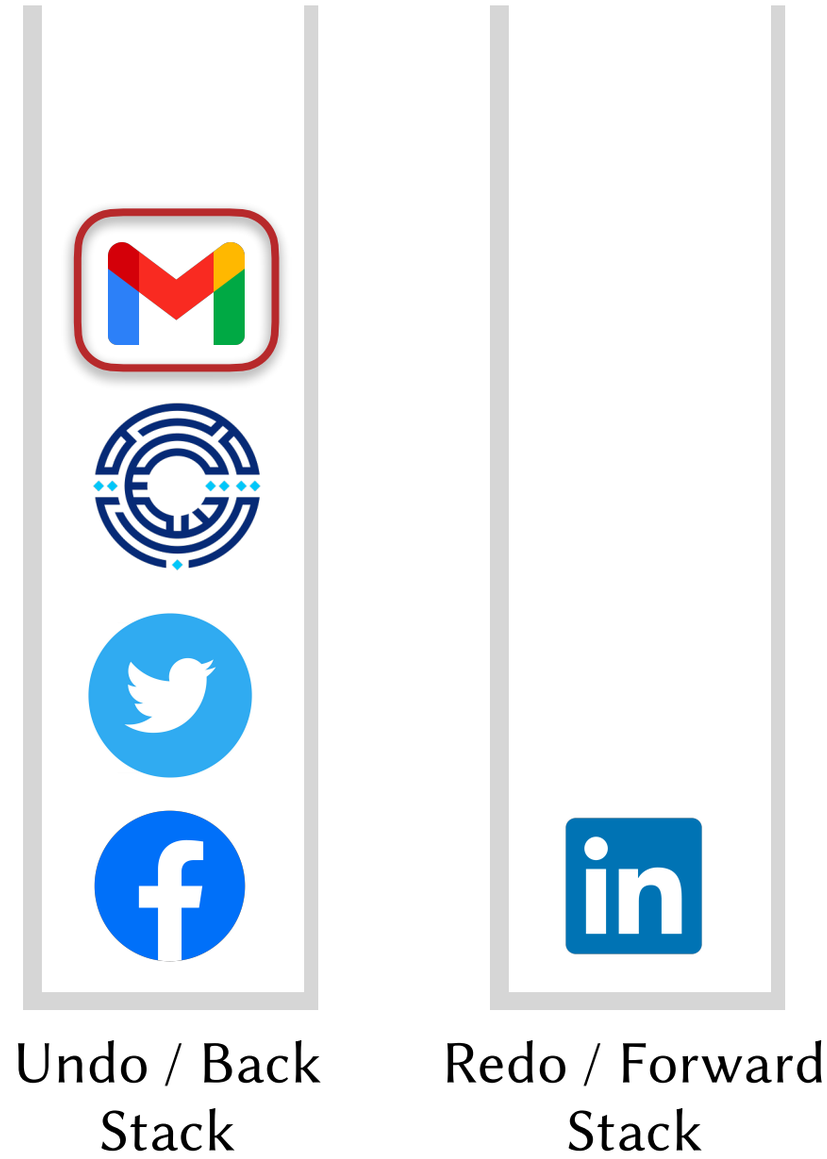



```
visit(page)

undo.push(page)
show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser



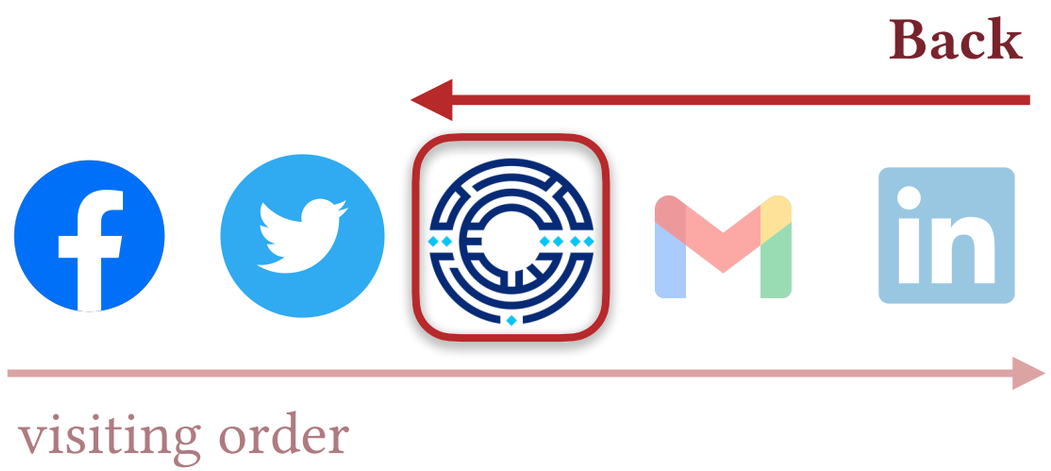


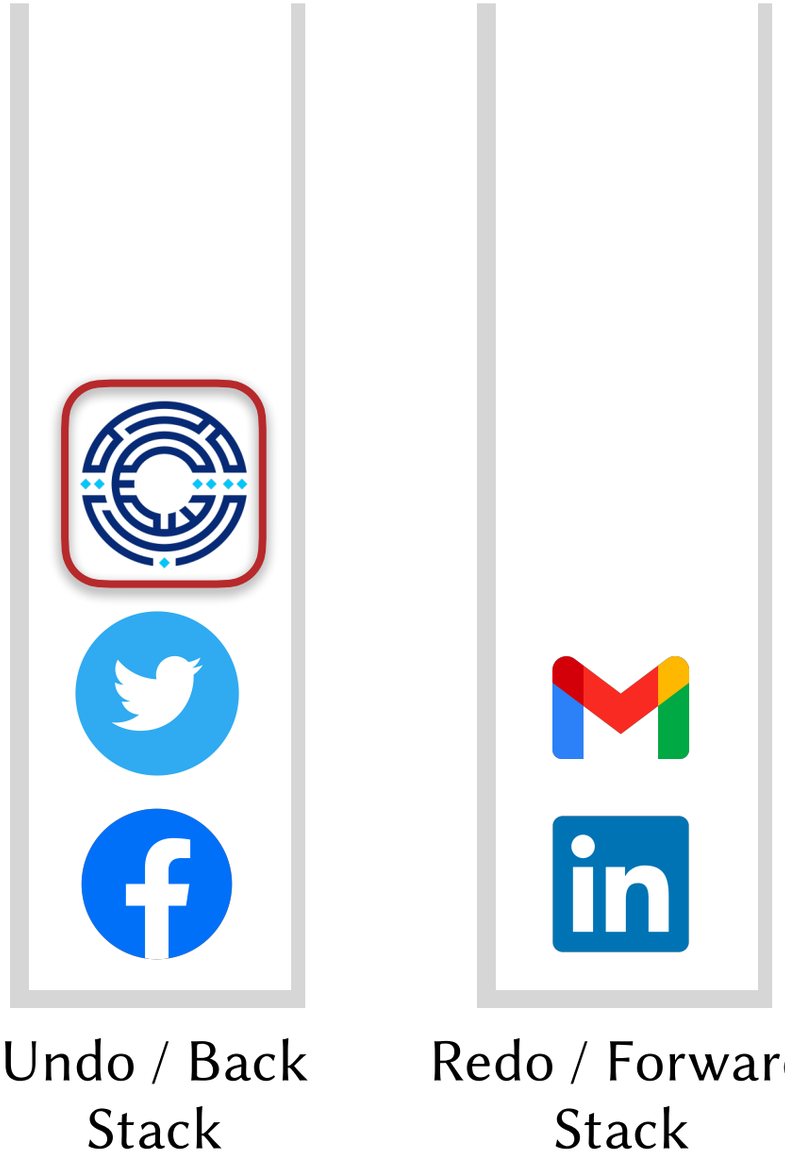

```
visit(page)
    undo.push(page)
    show(undo.top())
```

```
back()
    redo.push(undo.pop())
    show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser



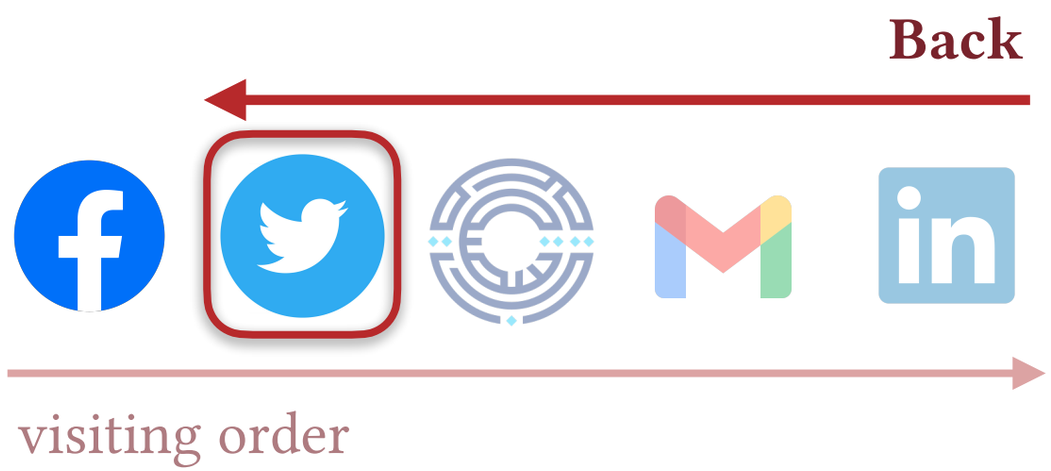


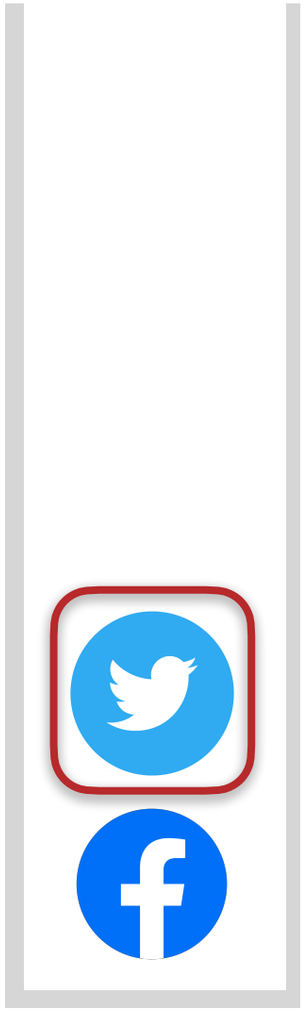

```
visit(page)
    undo.push(page)
    show(undo.top())
```

```
back()
    redo.push(undo.pop())
    show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser





Undo / Back Stack

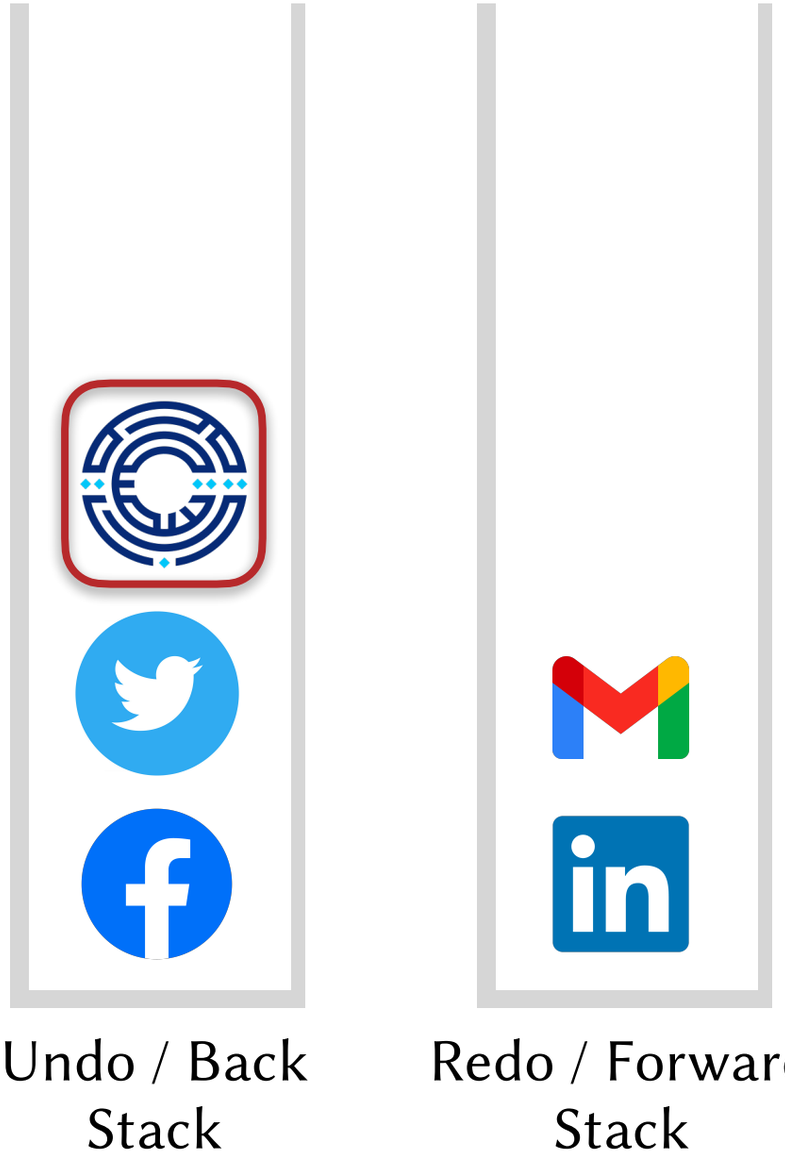
Redo / Forward Stack

```
visit(page)
    undo.push(page)
    show(undo.top())
```

```
back()
    redo.push(undo.pop())
    show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser




```
visit(page)
    undo.push(page)
    show(undo.top())
```

```
back()
    redo.push(undo.pop())
    show(undo.top())
```

```
forward()
    undo.push(redo.pop())
    show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. **Undo and Redo in a word processor**
   Back and Forward in a web browser

```
visit(page)
    undo.push(page)
    show(undo.top())
```

```
back()
    redo.push(undo.pop())
    show(undo.top())
```

```
forward()
    undo.push(redo.pop())
    show(undo.top())
```

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. Undo and Redo in a word processor
   Back and Forward in a web browser
4. **Balancing parentheses.**

   - `()[{}]([]{})` **VALID**
   - `((([{}]{[]{}}` **INVALID** (opened but not closed)
   - `[{}]))){[]{}}` **INVALID** (closed but not opened)
   - `[(]){()[}][]` **INVALID** (misbalanced)

Stacks (Applications)

1. Reversing sequences.
2. Managing function calls.
3. Undo and Redo in a word processor
   Back and Forward in a web browser
4. **Balancing parentheses.**

- `()[{}]([]{})` **VALID**
- `(([{}]{[]{}}` **INVALID**
- `[{}]))){[]{}}` **INVALID**
- `[(]){()[}][]` **INVALID**

CHECK_PARANTHS(string str)

```
for (int i=0; i<str.length(); i++) {
    if ( /* str[i] is (, {, or [ */ )
        stack.push(str[i]);
    else if (stack.is_empty() ||
            /* str[i] does not
             match stack.top() */ )
        return false;
    else
        stack.pop();
}
return stack.is_empty();
```

Try **tracing** the code!

Exercise

Which of the following are reasonable implementations for the Stack ADT?

- **Doubly-linked** list:
```
push(val) = add_to_head(val)
     pop() = remove_head()
```
- **Doubly-linked** list:
```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```
- **Singly-linked** list:
```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```
- **Singly-linked** list:
```
push(val) = add_to_head(val)
     pop() = remove_head()
```
- **Array** list:
```
push(val) = add_to_head(val)
     pop() = remove_head()
```
- **Array** list:
```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

Exercise

Which of the following are reasonable implementations for the Stack ADT?

✓ **Doubly-linked** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

✓ **Doubly-linked** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

✗ **Singly-linked** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

pop() = $O(n)$

✓ **Singly-linked** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

✗ **Array** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

pop() = $O(n)$
push() = $O(n)$

✓ **Array** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

Exercise

Which of the following are reasonable implementations for the Stack ADT?

✓ **Doubly-linked** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

✓ **Doubly-linked** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

✗ **Singly-linked** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

pop() = $O(n)$

😁 **Singly-linked** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

Uses less memory than DLList & avoids array resizing!

✗ **Array** list:

```
push(val) = add_to_head(val)
     pop() = remove_head()
```

pop() = $O(n)$
push() = $O(n)$

😁 **Array** list:

```
push(val) = add_to_tail(val)
     pop() = remove_tail()
```

Makes good use of cache memory and avoids creating and deleting nodes.

Stack as a DLL (Implementation)

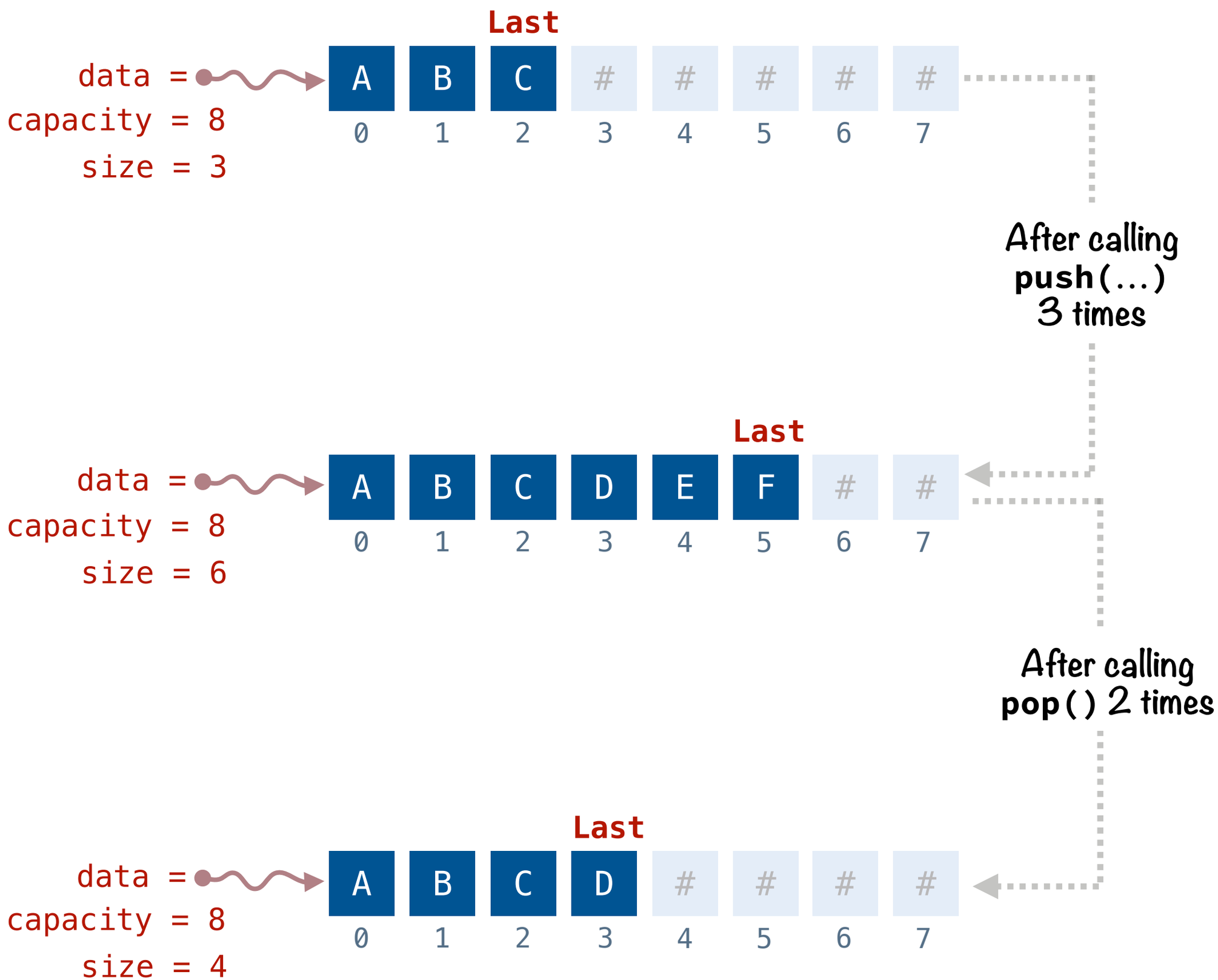
```
template <class T>
class Stack {
private:
    DLList<T> list;

public:
    bool is_empty() const     { return list.is_empty(); }
    void clear()              { list.clear(); }
    void push(const T& val)   { list.add_to_head(val); }
    T top() const             { return list.head_val(); }

    T pop() {
        T val temp = list.head_val();
        list.remove_head();
        return temp;
    }
};
```

delegate all the work to this object

throws an exception if the list is empty!

No need to implement a default constructor, a copy constructor, or a destructor, or overload the assignment operator because this was already done in class `DLList`!

Stack as an **Array** (Implementation)



Exercise

Which of the following are reasonable implementation for function **`clear()`**?

A.
```
void clear() { last = -1; }
```

B.
```
void clear() {
    last = -1;
    delete [] data;
}
```

C.
```
void clear() {
    last = -1;
    delete [] data;
    data = new data[capacity];
}
```

D. None of the above.

Exercise

Which of the following are reasonable implementation for function **`clear()`**?

✓
```
void clear() { last = -1; }
```

The next `push(...)` statement moves last to index `0`

✗
```
void clear() {
    last = -1;
    delete [] data;
}
```

The array is not usable after this statement.

✗
```
void clear() {
    last = -1;
    delete [] data;
    data = new data[capacity];
}
```

Unnecessarily removes the array and crates a new one with the same capacity.

D. None of the above.

Queues (a new ADT)

Queue

```
void enqueue(const T& val)
   T dequeue()
   T get_first() const
   T get_last()  const
```

add `val` to the collection of elements

remove the first added element

return the first added element

return the last added element

! Elements cannot be accessed or modified in any other way

First come first serve

FIFO: Fast In First Out

~~**LILO:** Last In Last Out~~

Queues (a new ADT)

Queue

```
void enqueue(const T& val)
   T dequeue()
   T get_first() const
   T get_last()  const
```

add `val` to the collection of elements

remove the first added element

return the first added element

return the last added element

! Elements cannot be accessed or modified in any other way

Applications.

1. Ticketing systems.
2. Printer queue.
3. Website requests queue.
4. Video streaming buffer.

etc. (too many to be listed)

FIFO: Fast In First Out

~~**LILO:** Last In Last Out~~

Exercise

Which of the following are reasonable implementations for the Queue ADT?

- **Doubly-linked** list:
```
enqueue(val) = add_to_head(val)
   dequeue() = remove_tail()
```
- **Doubly-linked** list:
```
enqueue(val) = add_to_tail(val)
   dequeue() = remove_head()
```
- **Singly-linked** list:
```
enqueue(val) = add_to_tail(val)
   dequeue() = remove_head()
```
- **Singly-linked** list:
```
enqueue(val) = add_to_head(val)
   dequeue() = remove_tail()
```
- **Array** list:
```
enqueue(val) = add_to_head(val)
   dequeue() = remove_tail()
```
- **Array** list:
```
enqueue(val) = add_to_tail(val)
   dequeue() = remove_head()
```

Exercise

Which of the following are reasonable implementations for the Queue ADT?

✓ **Doubly-linked** list:

```
enqueue(val) = add_to_head(val)
dequeue() = remove_tail()
```

✓ **Doubly-linked** list:

```
enqueue(val) = add_to_tail(val)
dequeue() = remove_head()
```

✓ **Singly-linked** list:

```
enqueue(val) = add_to_tail(val)
dequeue() = remove_head()
```

✗ **Singly-linked** list:

```
enqueue(val) = add_to_head(val)
dequeue() = remove_tail()
```

← $O(n)$

✗ **Array** list:

```
enqueue(val) = add_to_head(val)
dequeue() = remove_tail()
```

← $O(n)$

✗ **Array** list:

```
enqueue(val) = add_to_tail(val)
dequeue() = remove_head()
```

← $O(n)$

A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.

A Queue As An Array

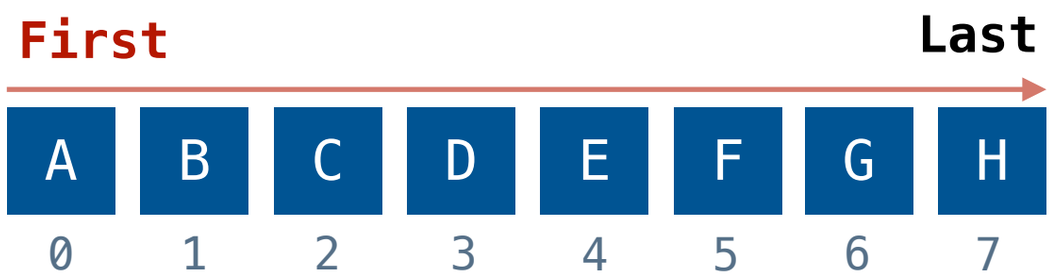
Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.

A full queue



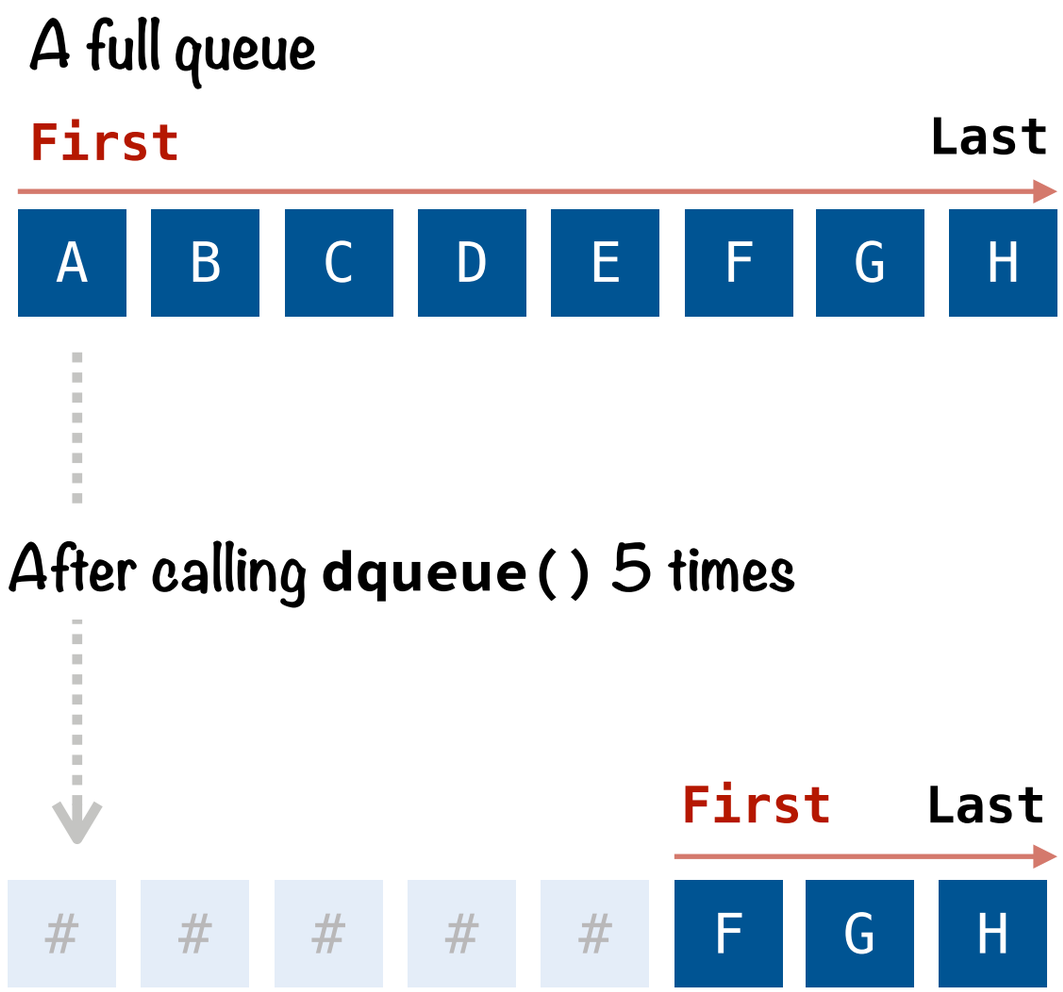
A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.



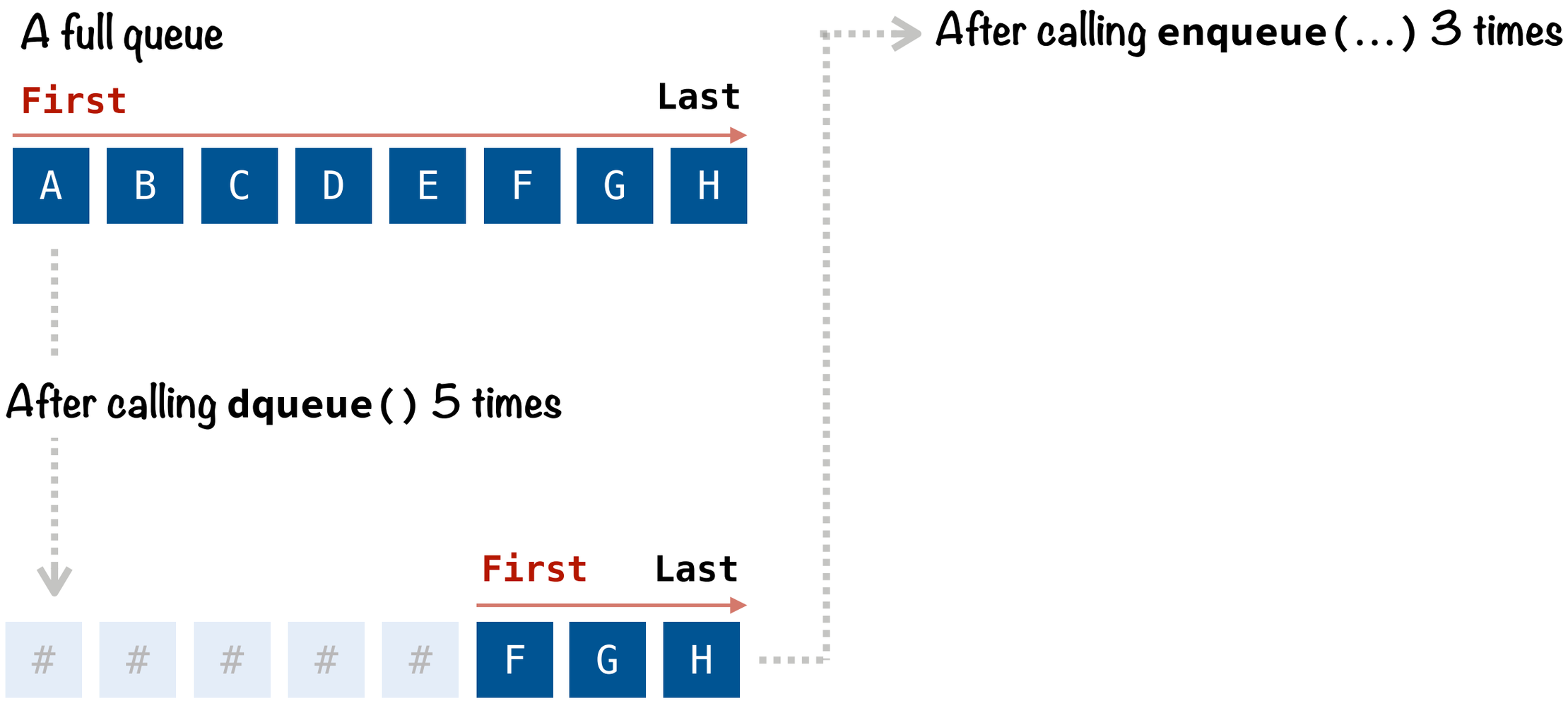
A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.



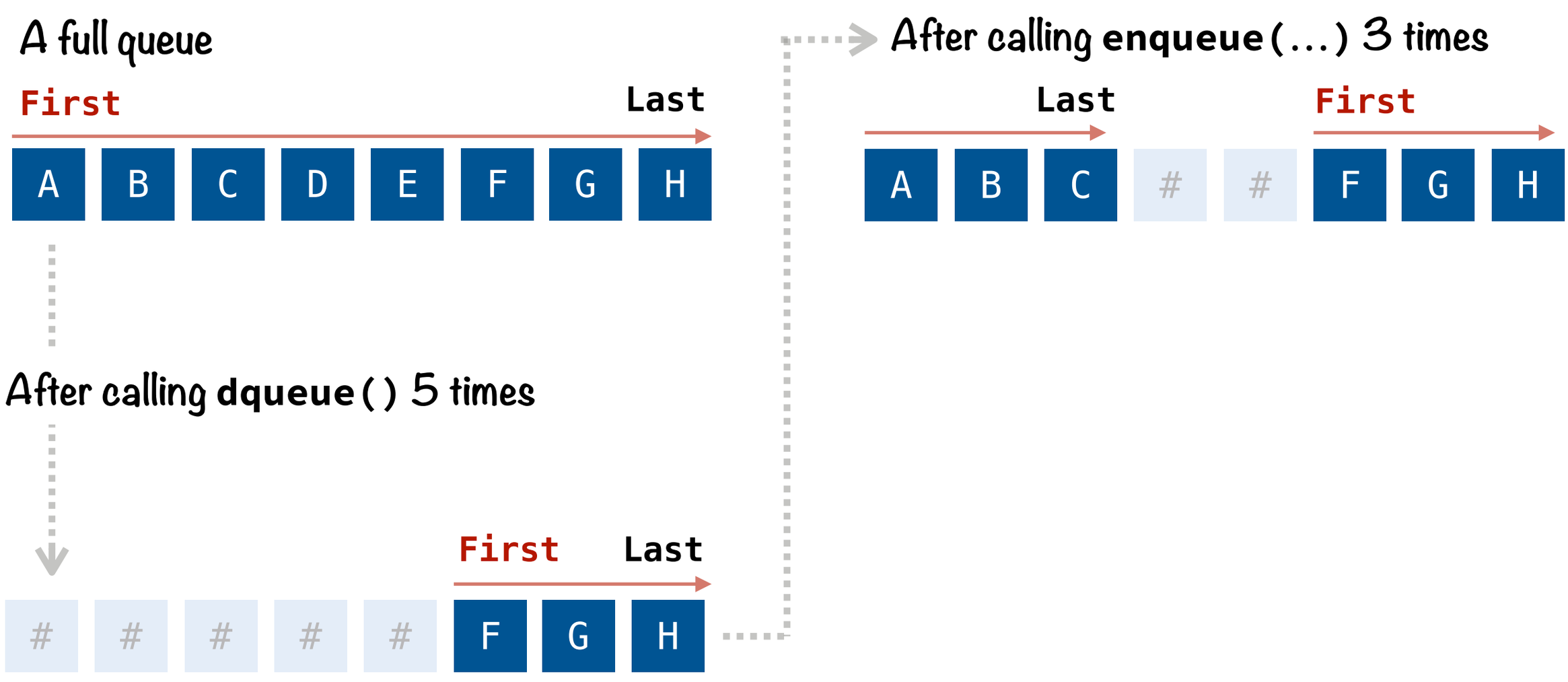
A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.



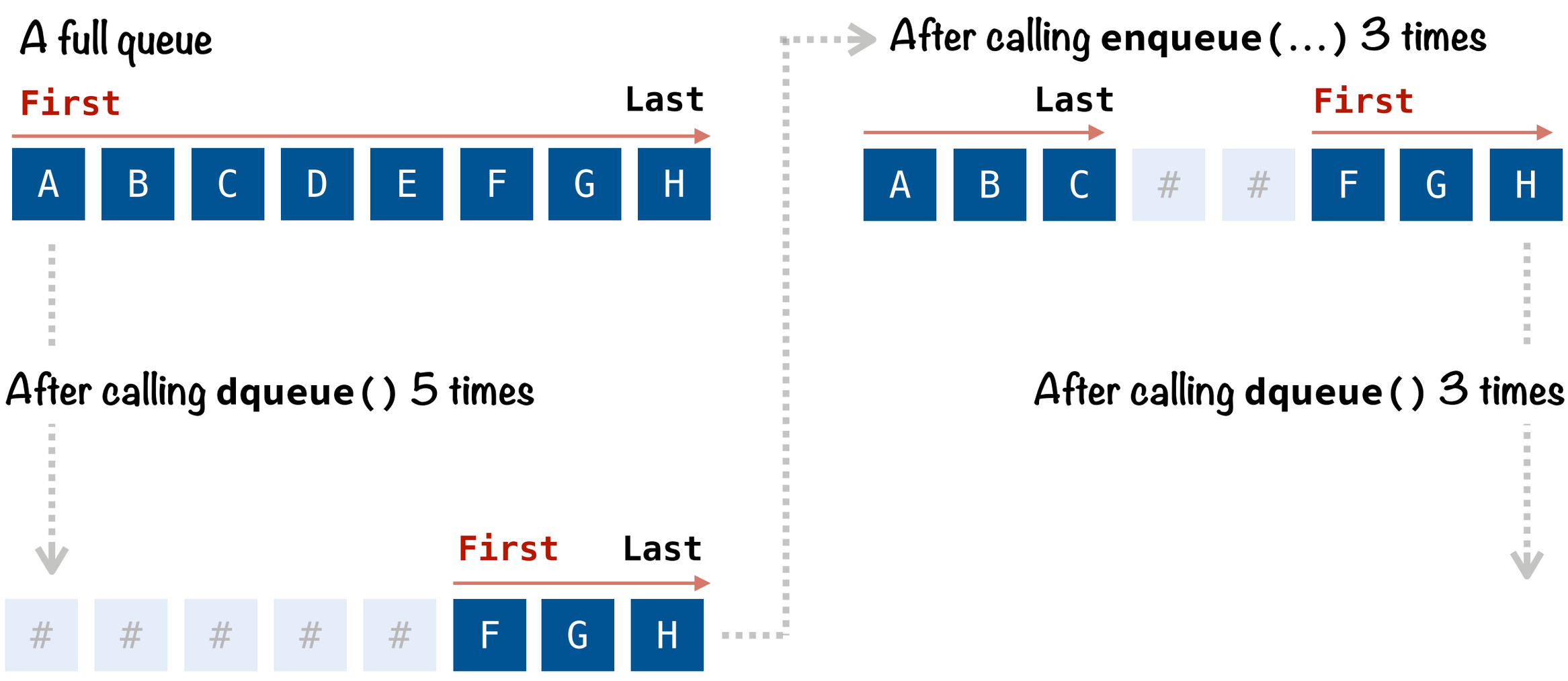
A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.



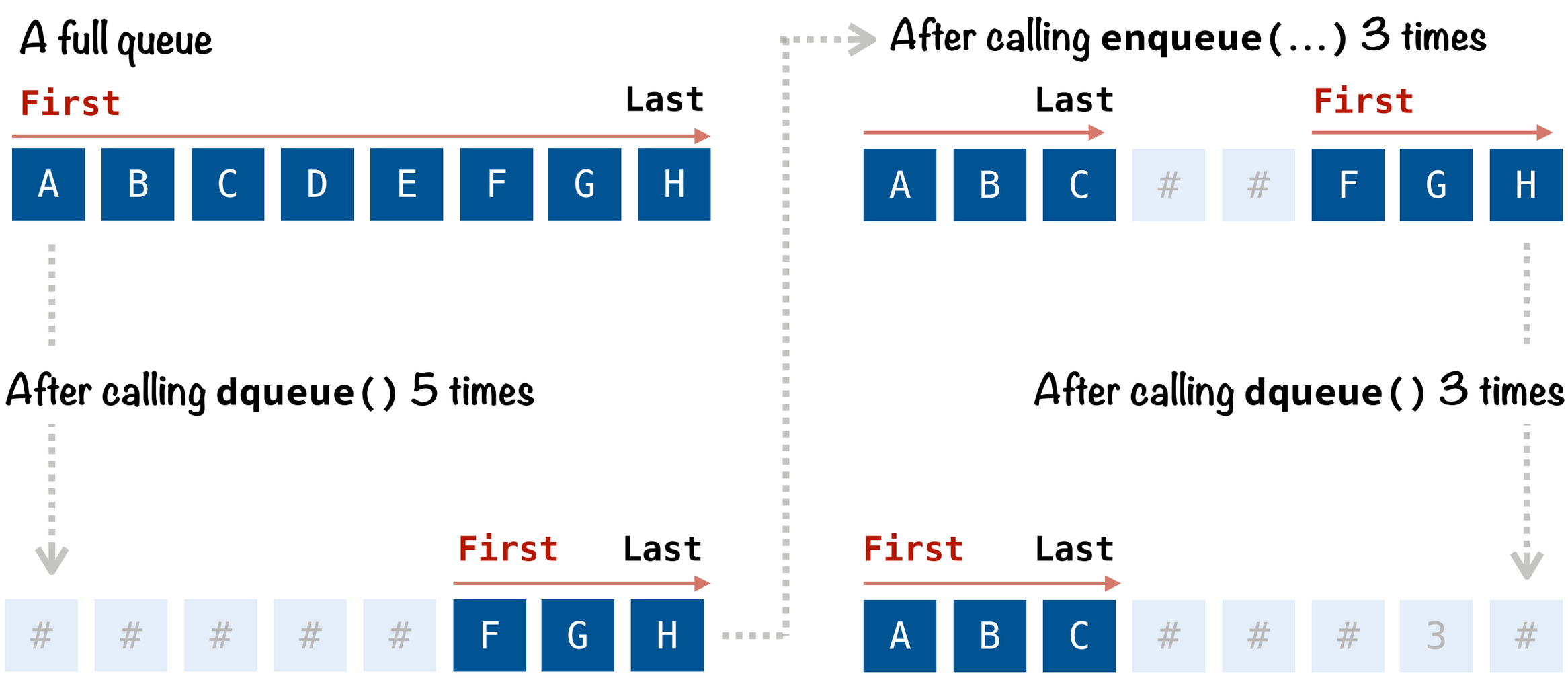
A Queue As An Array

Problem.

- A Queue must modify the head (either to remove or to add).
- Adding to or removing from the head of a regular array list requires shifting (costs $O(n)$)

Solution.

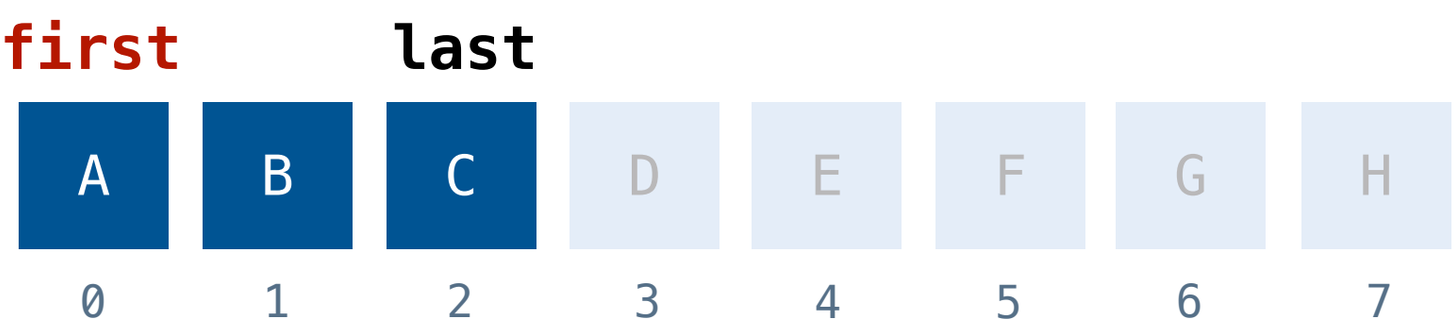
- Avoid shifting: Store the index of where the elements start in the queue and where they end.
- Wrap-around if needed.



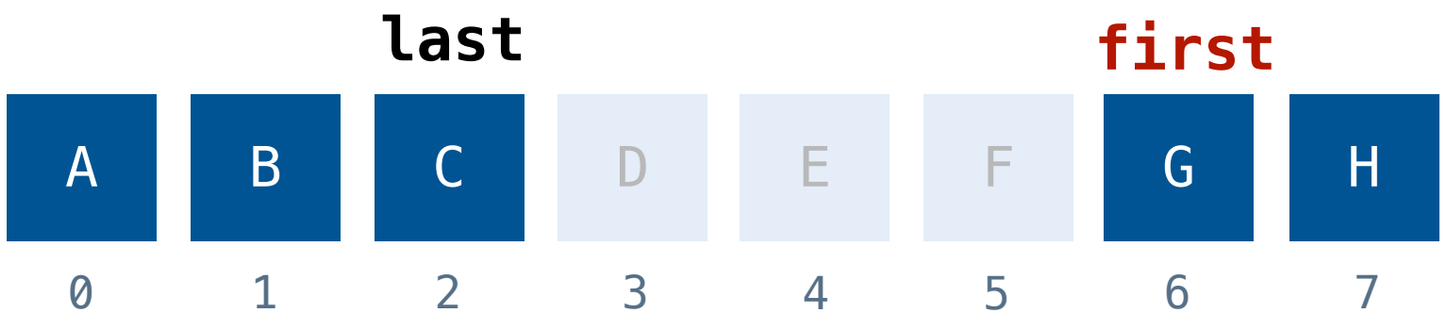
Queues as Arrays (data members)

first = -1
last = -1
capacity = 8
size = 0

capacity = 8
size = 3

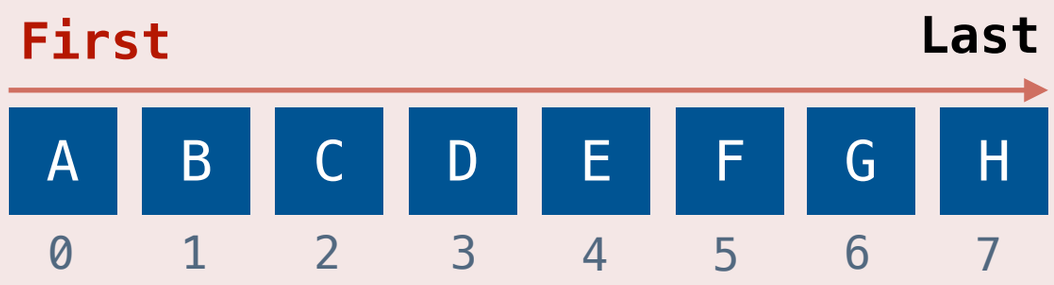


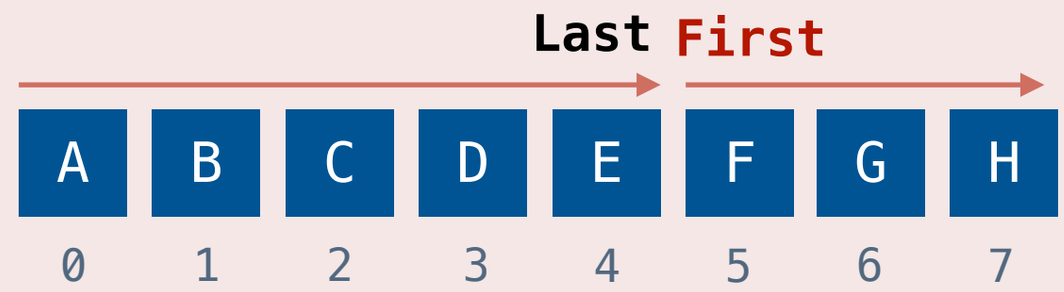
capacity = 8
size = 5

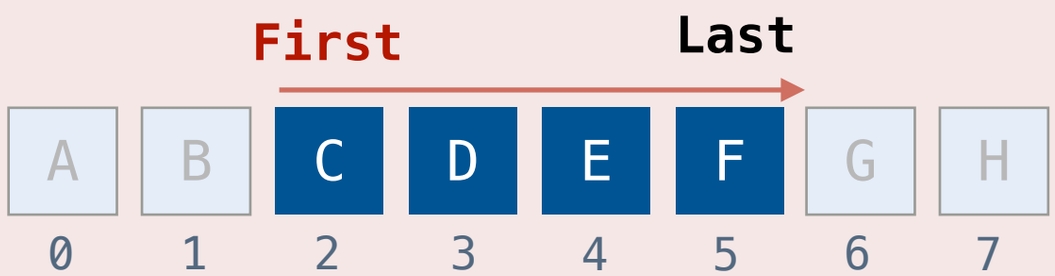


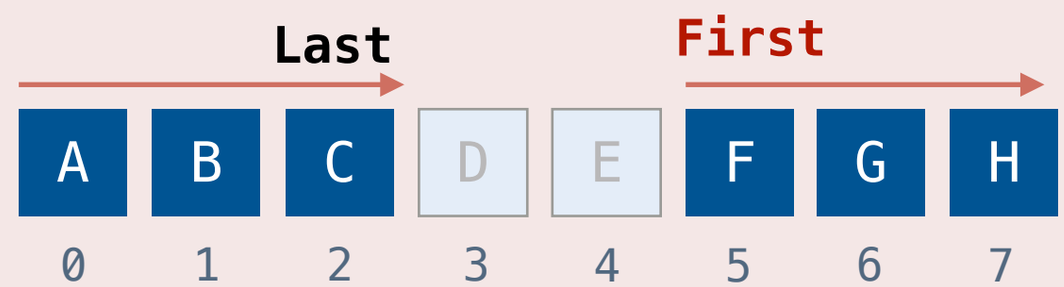
Exercise

Implement function **`is_full()`**



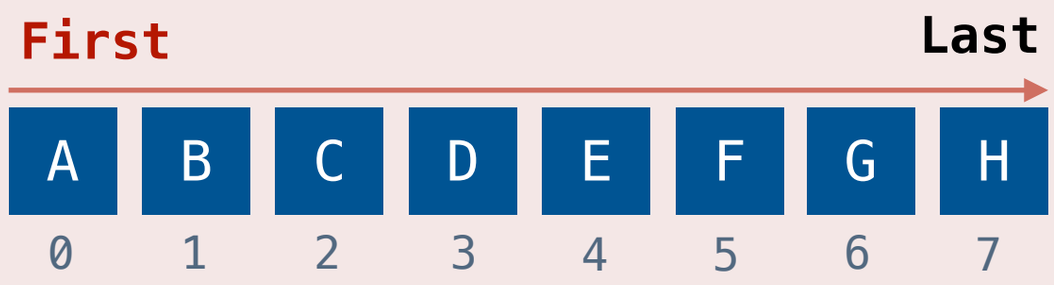


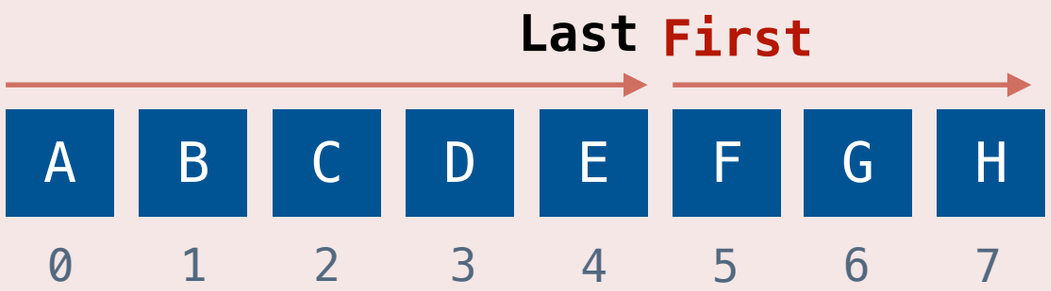


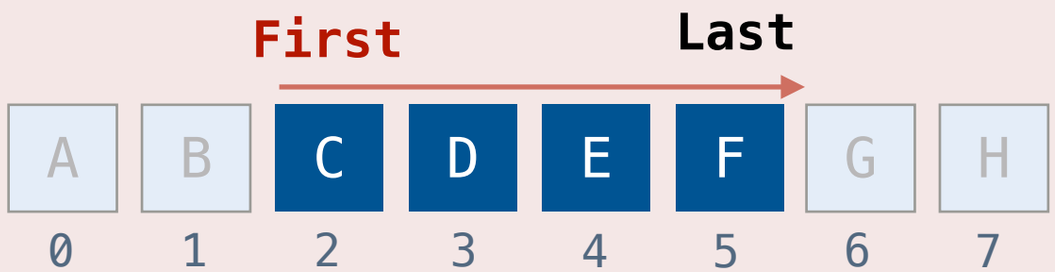


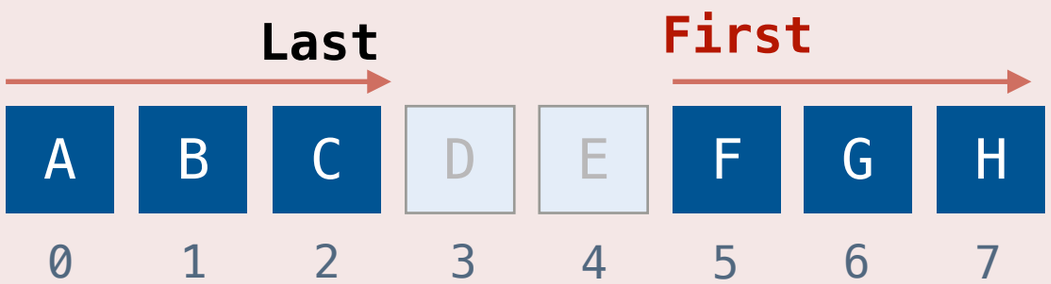
Exercise

Implement function **`is_full()`**









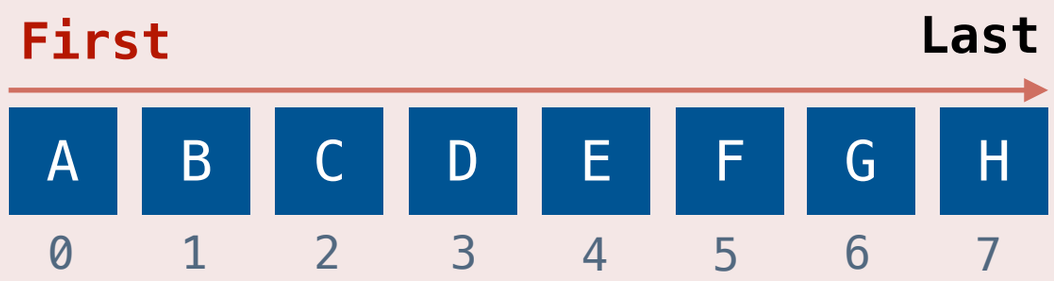

```
bool is_full() const {
    return (last+1) % capacity == first;
}
```

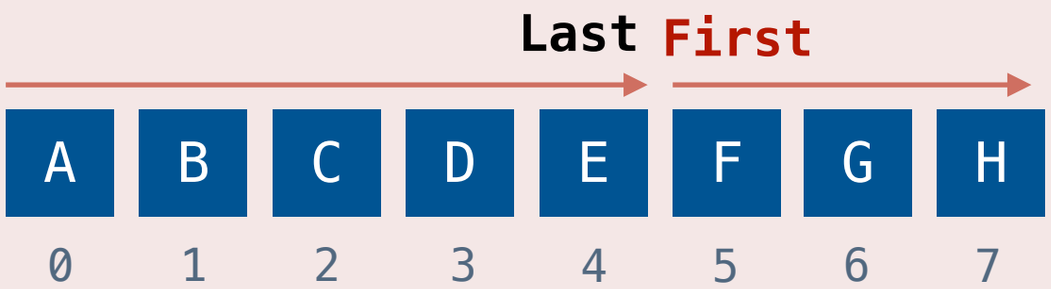
```
bool is_full() const {
    return size == cap;
}
```

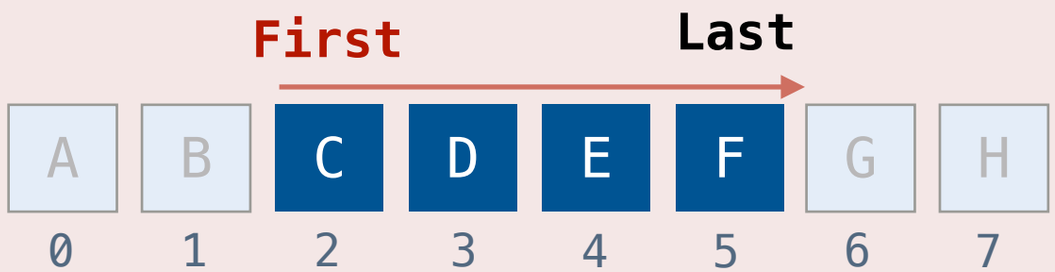
```
bool is_full() const {
    return
    last-first+1 == capacity ||
          last+1 == first;
}
```

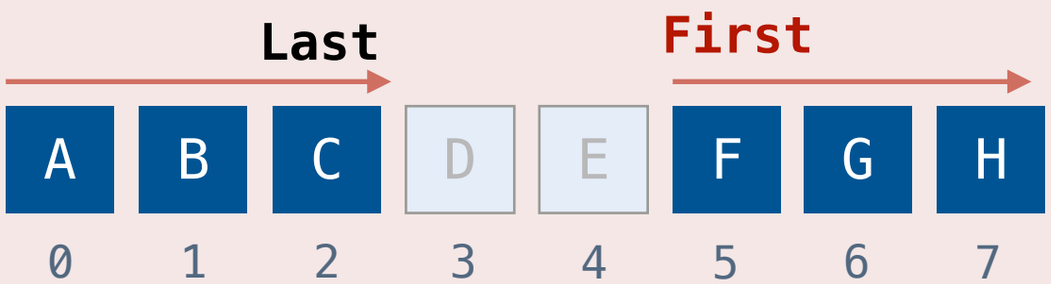
Exercise

Implement function **`is_full()`**










```
bool is_full() const {
    return (last+1) % capacity == first;
}
```

```
bool is_full() const {
    return size == cap;
}
```

```
bool is_full() const {
   return
   last-first+1 == capacity ||
         last+1 == first;
}
```

Buggy code!

What if the queue is empty and capacity = 1?

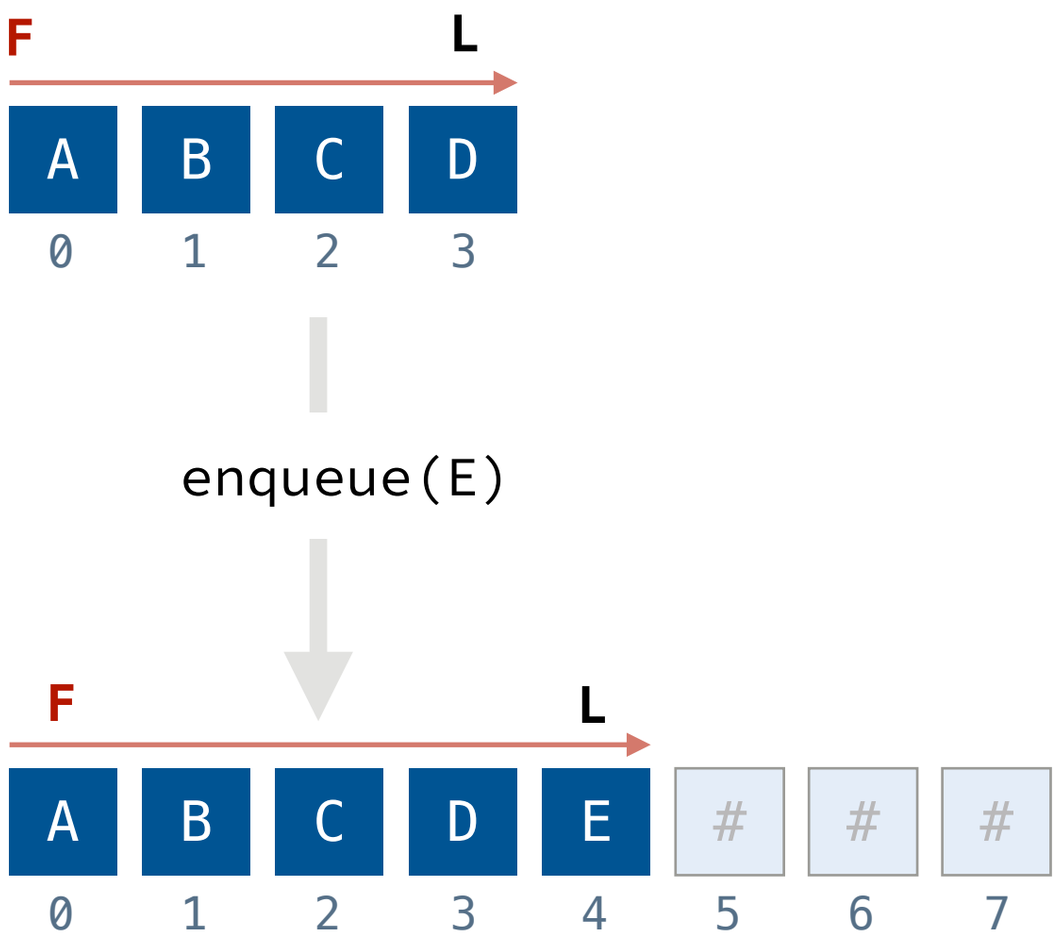
Queues (Implementation Plan)

Resize up if full

```
last = (last + 1) % capacity;
data[last] = val;
```

Queue

```
void enqueue(const T& val)

    T dequeue()
```



Queues (Implementation Plan)

Queue

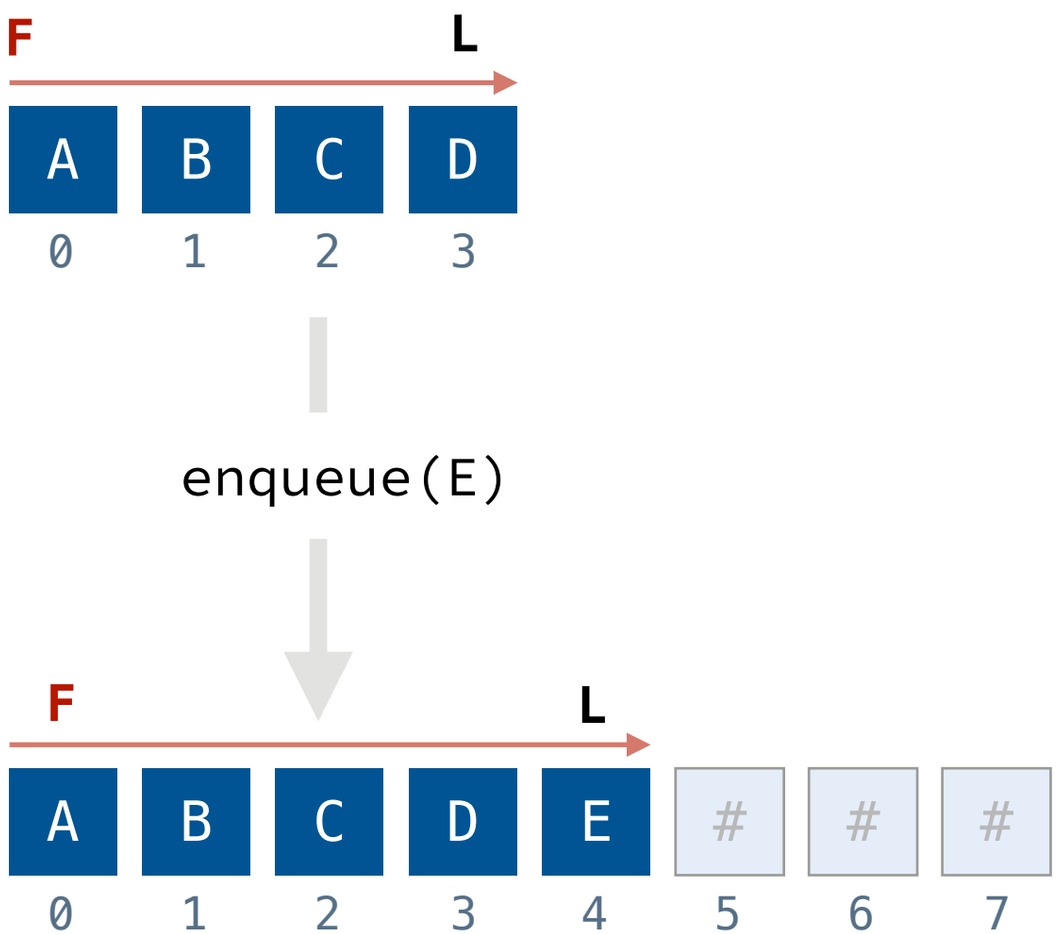
```
void enqueue(const T& val)

   T dequeue()
```

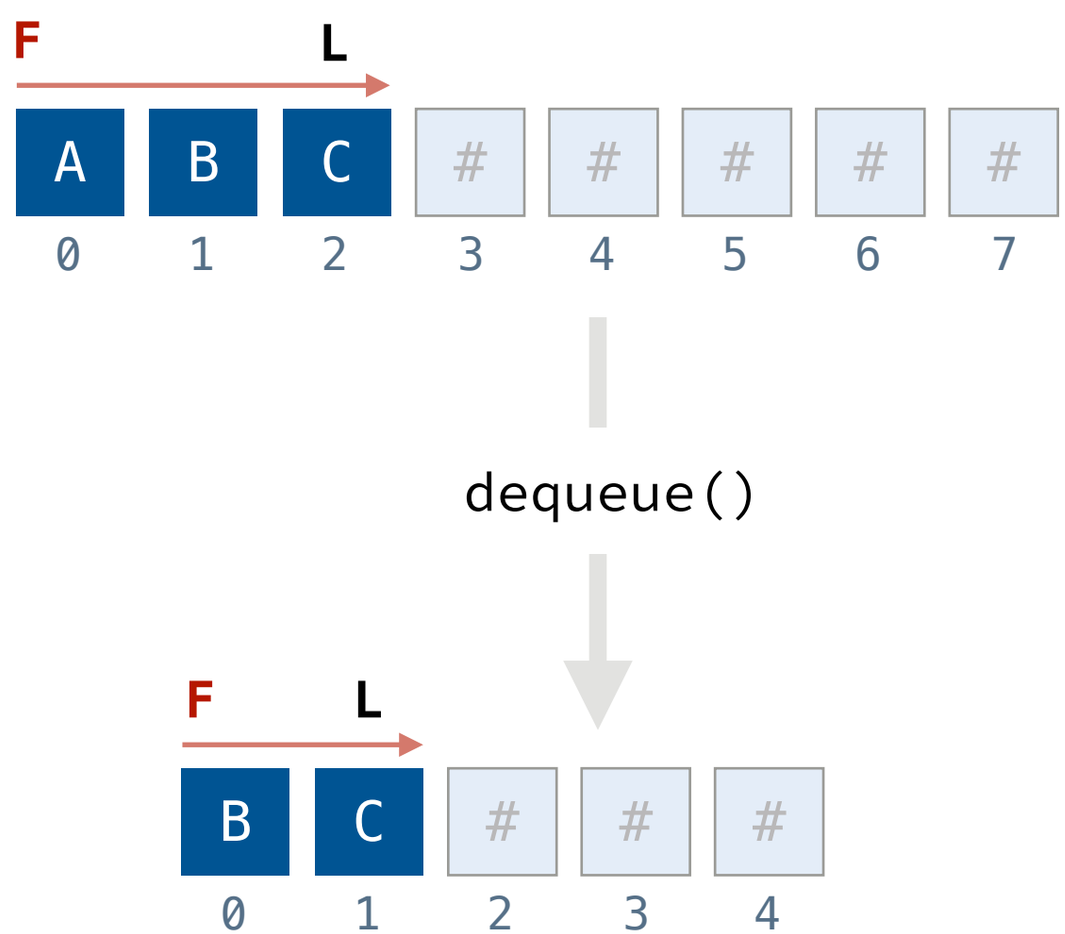
Resize up if full

```
last = (last + 1) % capacity;
data[last] = val;
```

```
first = (first + 1) % capacity;
```

Resize down if $\frac{1}{4}$ full





Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```

Choose the *best* answer.

A. $O(n^2)$

B. $O(n \log n)$

C. $O(n)$

D. $O(\log n)$

Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```

Choose the *best* answer.

A. $O(n^2)$

B. $O(n \log n)$

Too pessimistic!

$O(n)$

Correct!

D. $O(\log n)$

Incorrect

Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```

x 1

Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

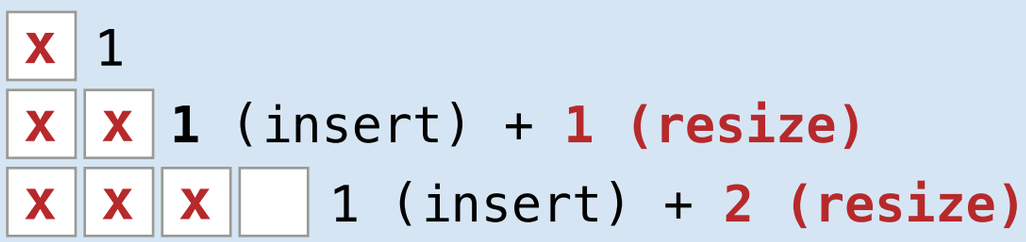
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```

- [x] 1
- [x] [x] **1** (insert) + **1 (resize)**

Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

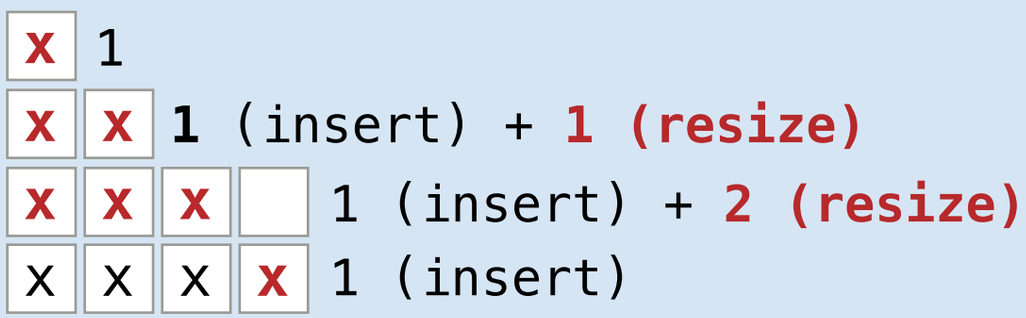
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

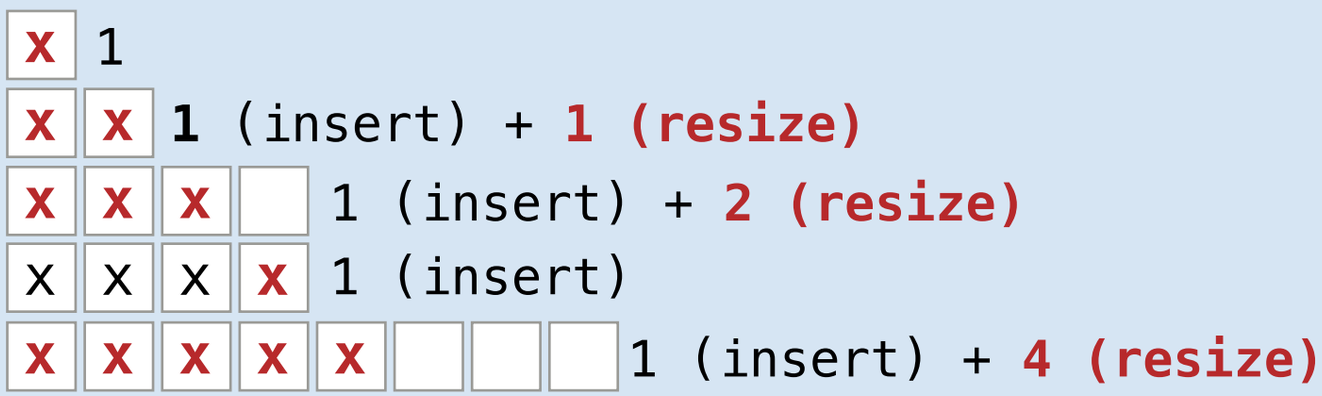
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

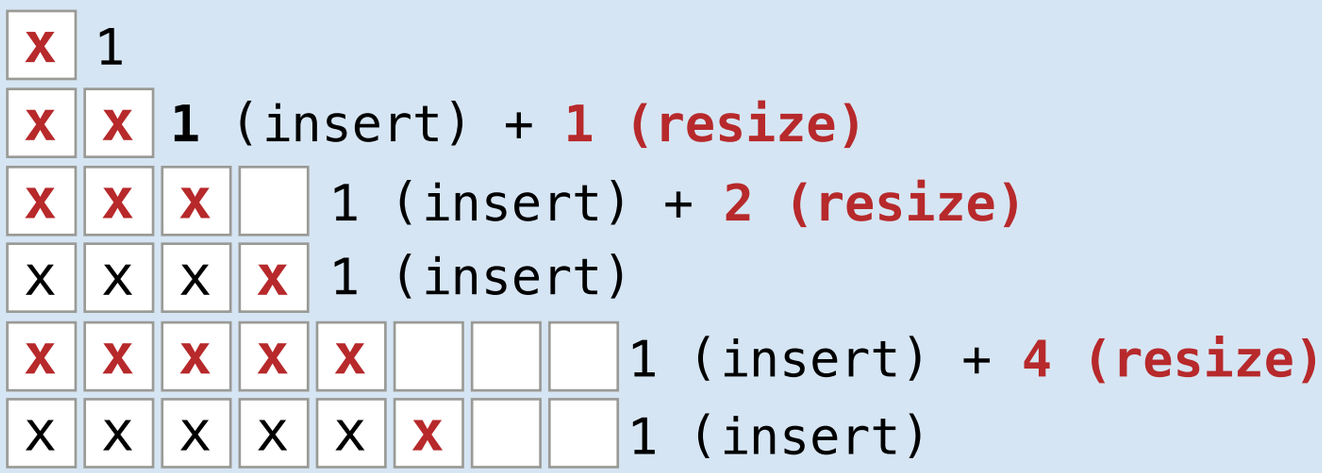
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

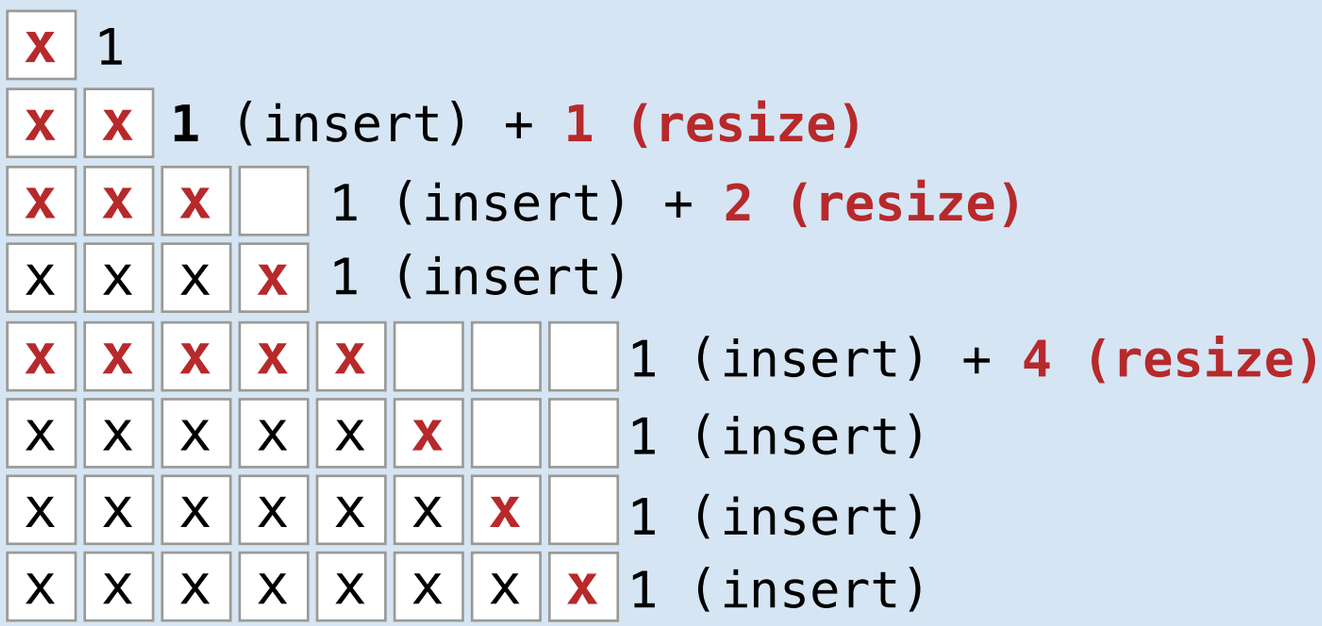
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

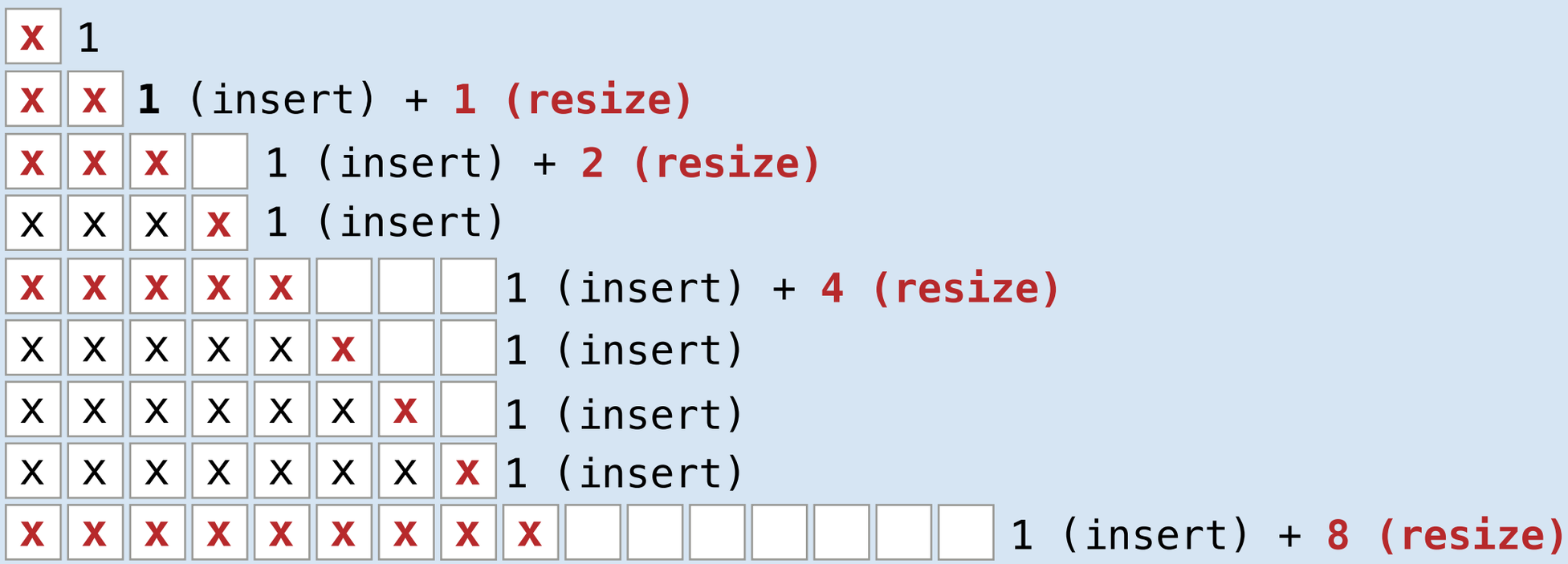
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

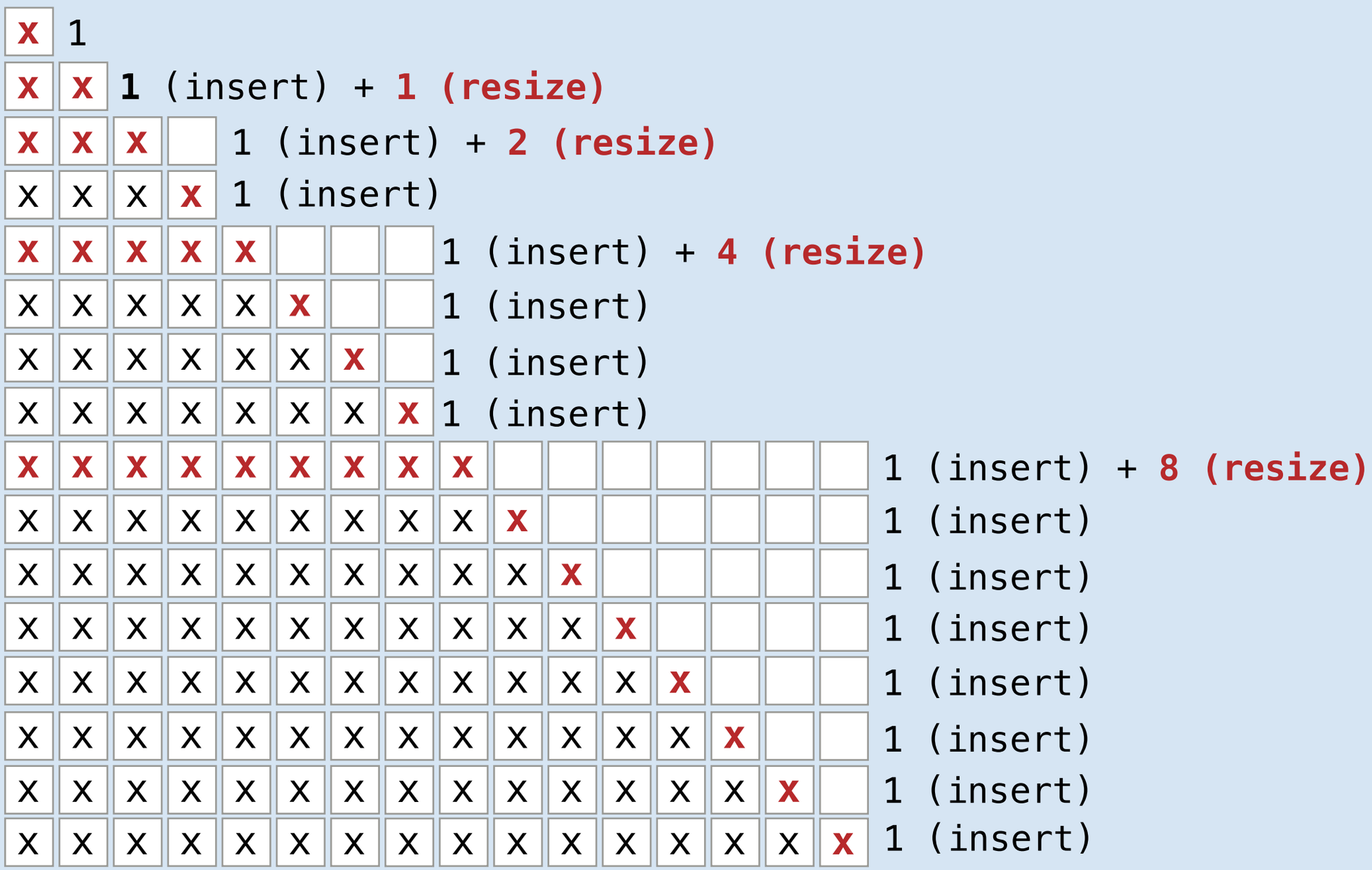
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

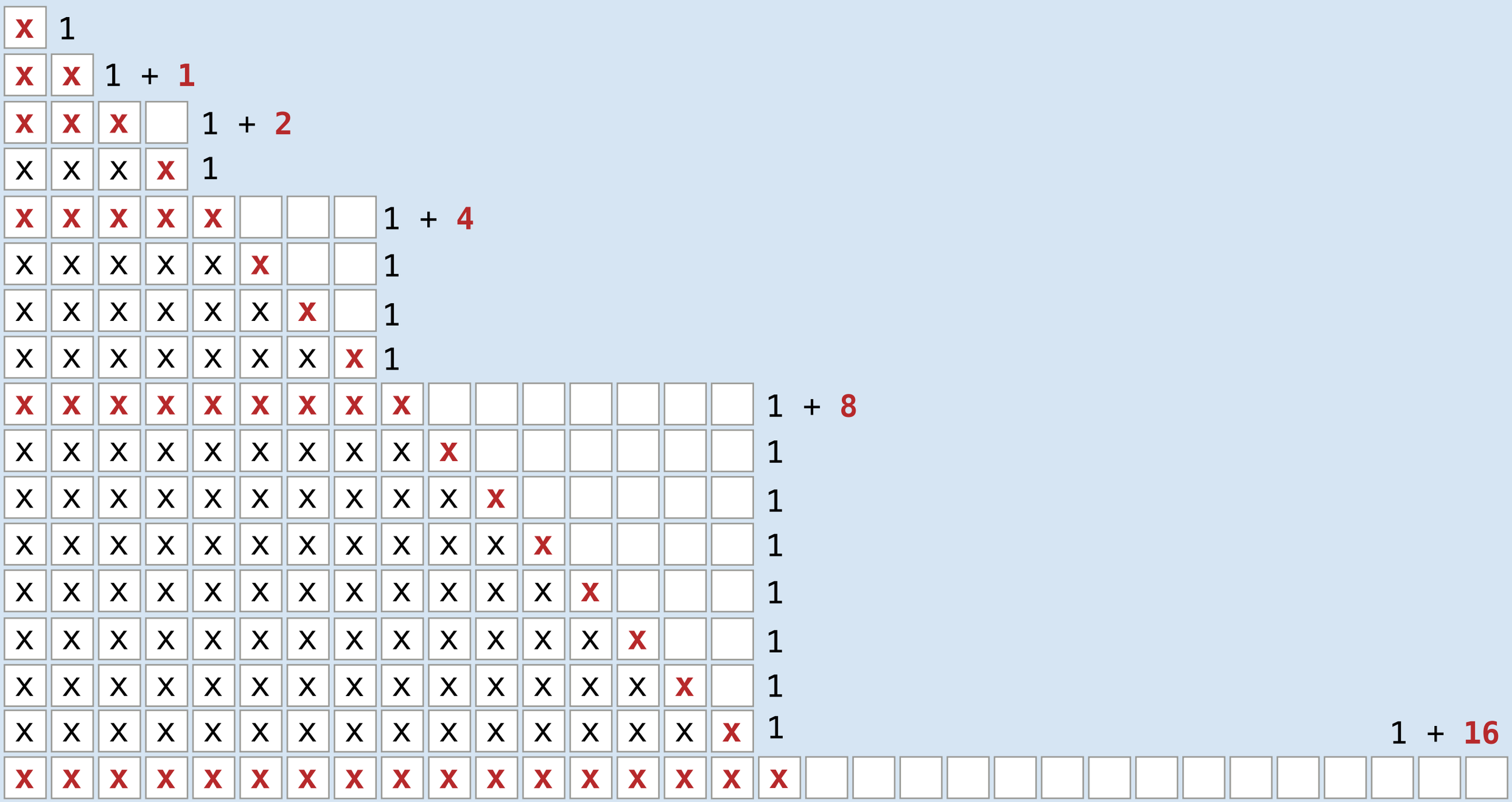
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

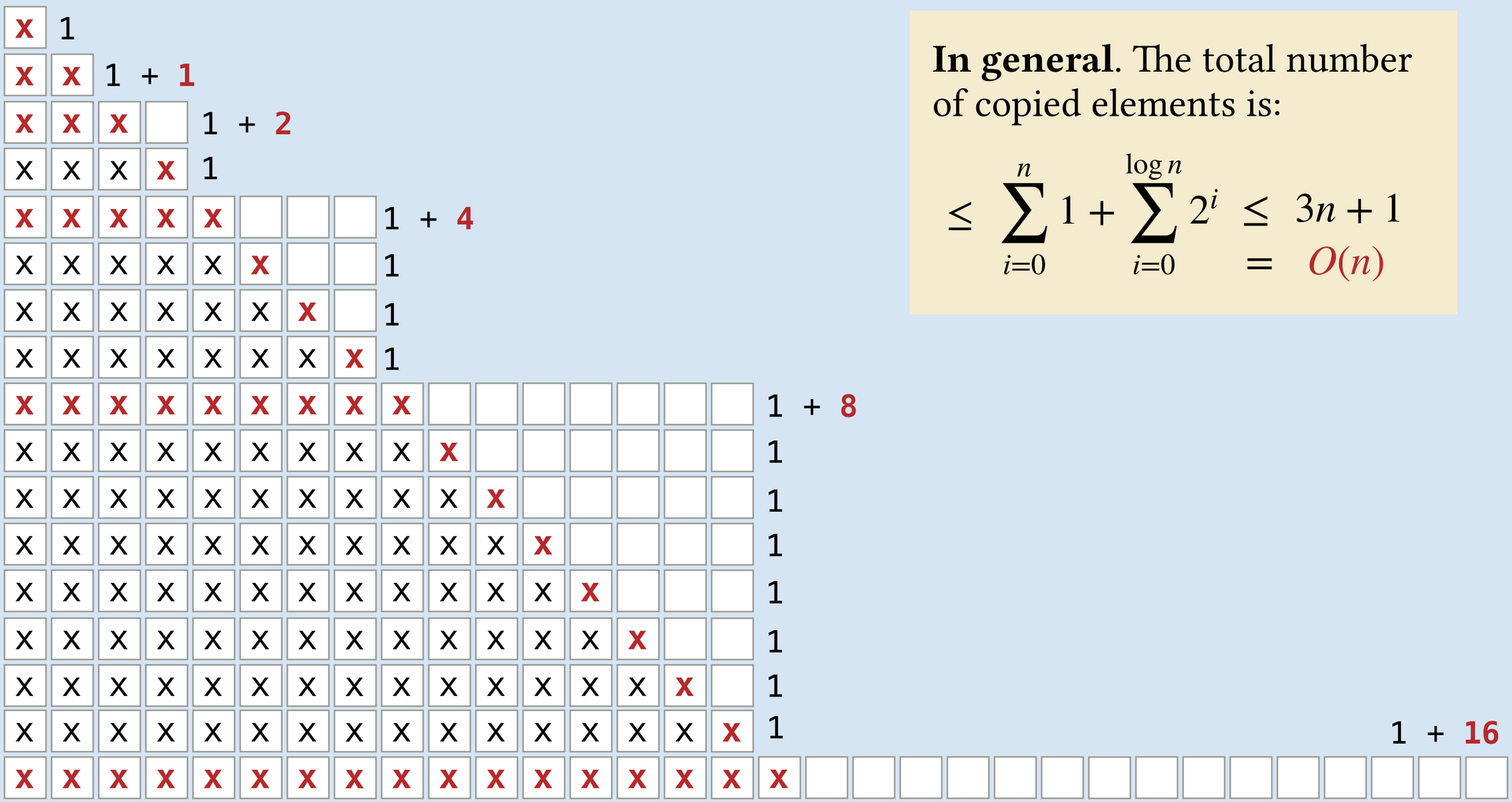
```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



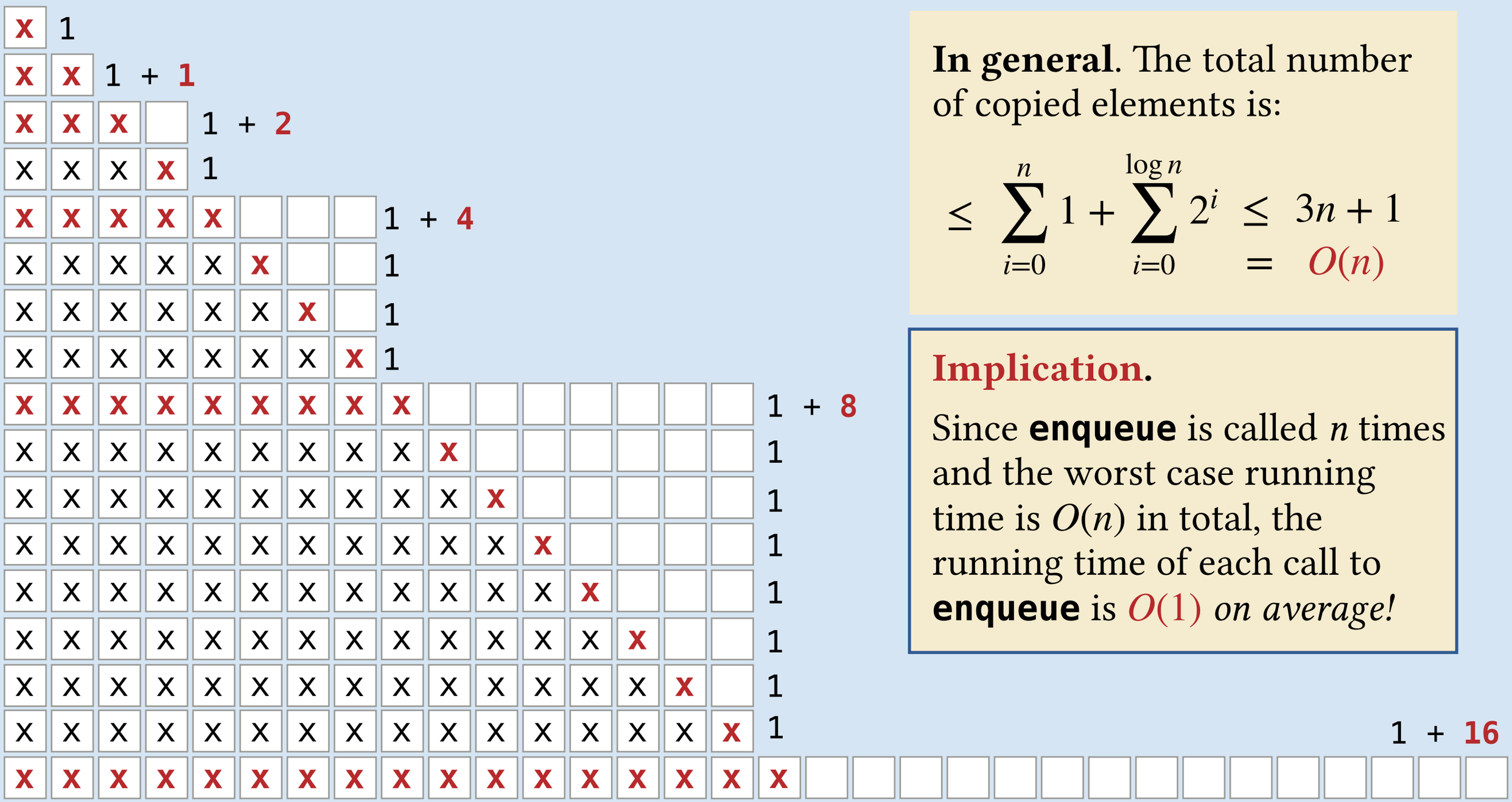
In general. The total number of copied elements is:

$$\leq \sum_{i=0}^{n} 1 + \sum_{i=0}^{\log n} 2^i \leq 3n + 1 = O(n)$$

Exercise

What is the running time of the following code? Assume the queue is initially of size 1.

```
for (int i = 0; i < n; i++)
    myQueue.enqueue(i);
```



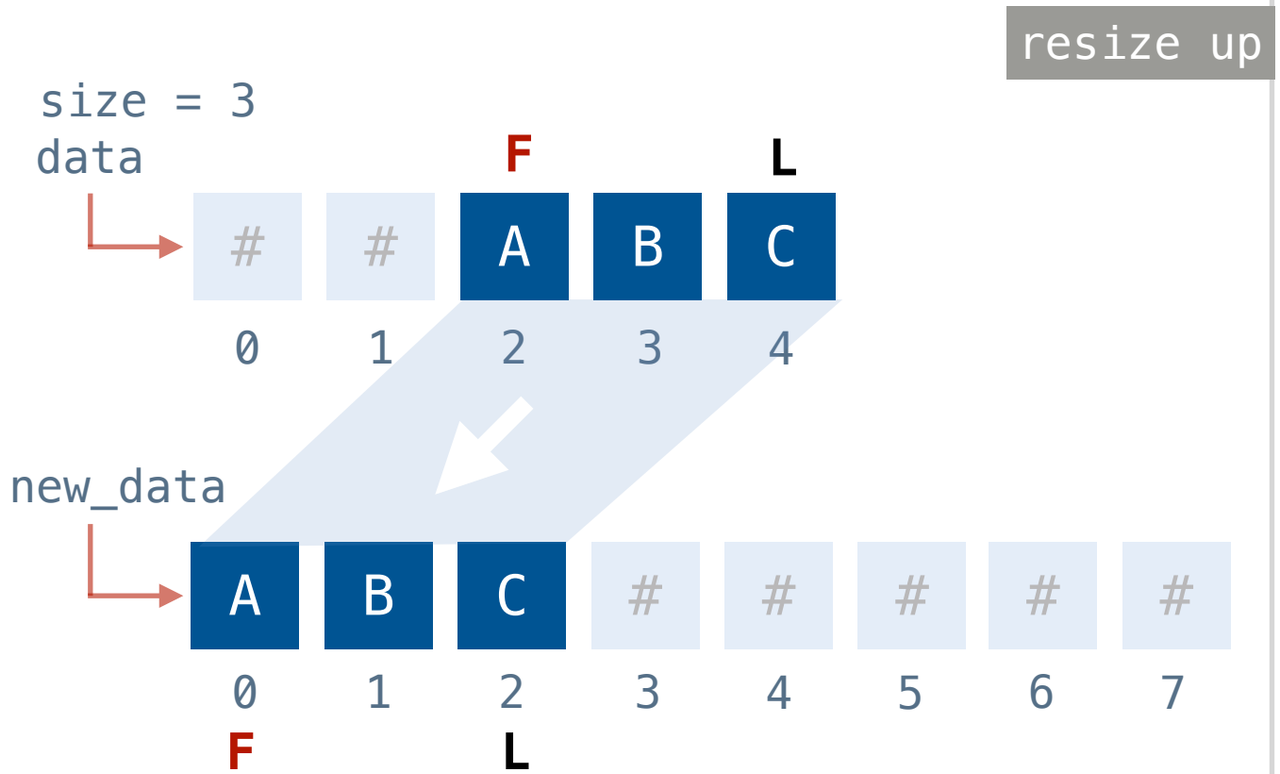
In general. The total number of copied elements is:

$$\leq \sum_{i=0}^{n} 1 + \sum_{i=0}^{\log n} 2^i \leq 3n + 1 = O(n)$$

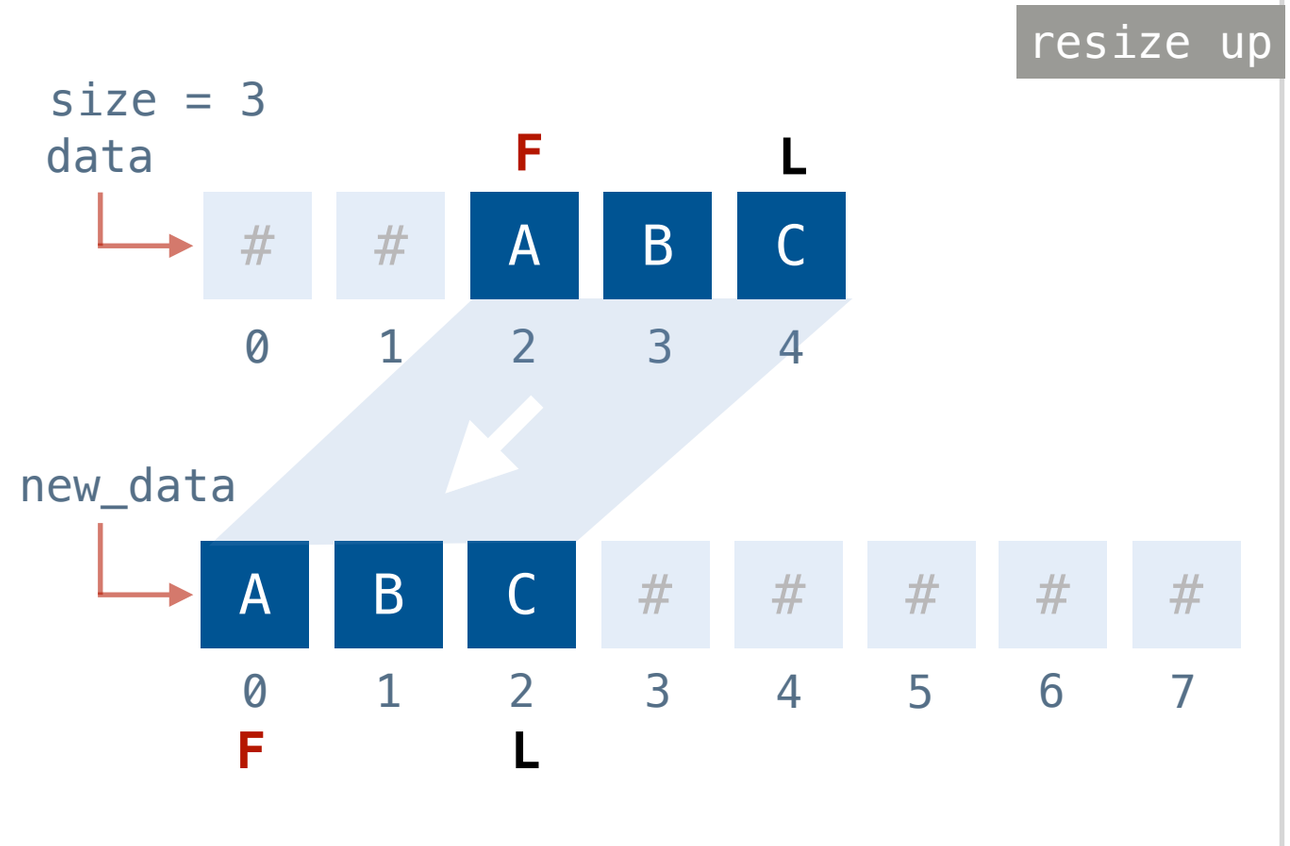
Implication.

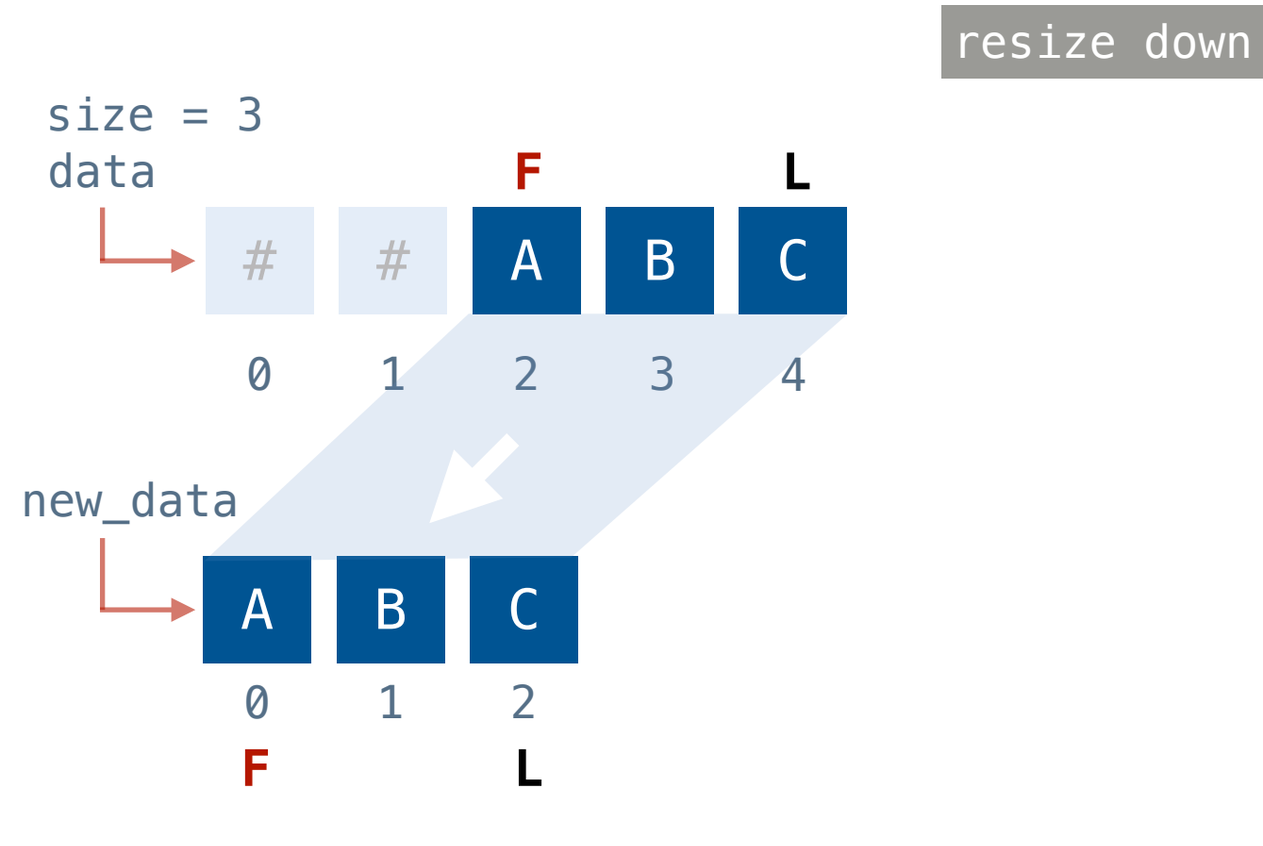
Since **enqueue** is called n times and the worst case running time is $O(n)$ in total, the running time of each call to **enqueue** is $O(1)$ *on average!*

Resizing The Queue (resize and reorder the elements)

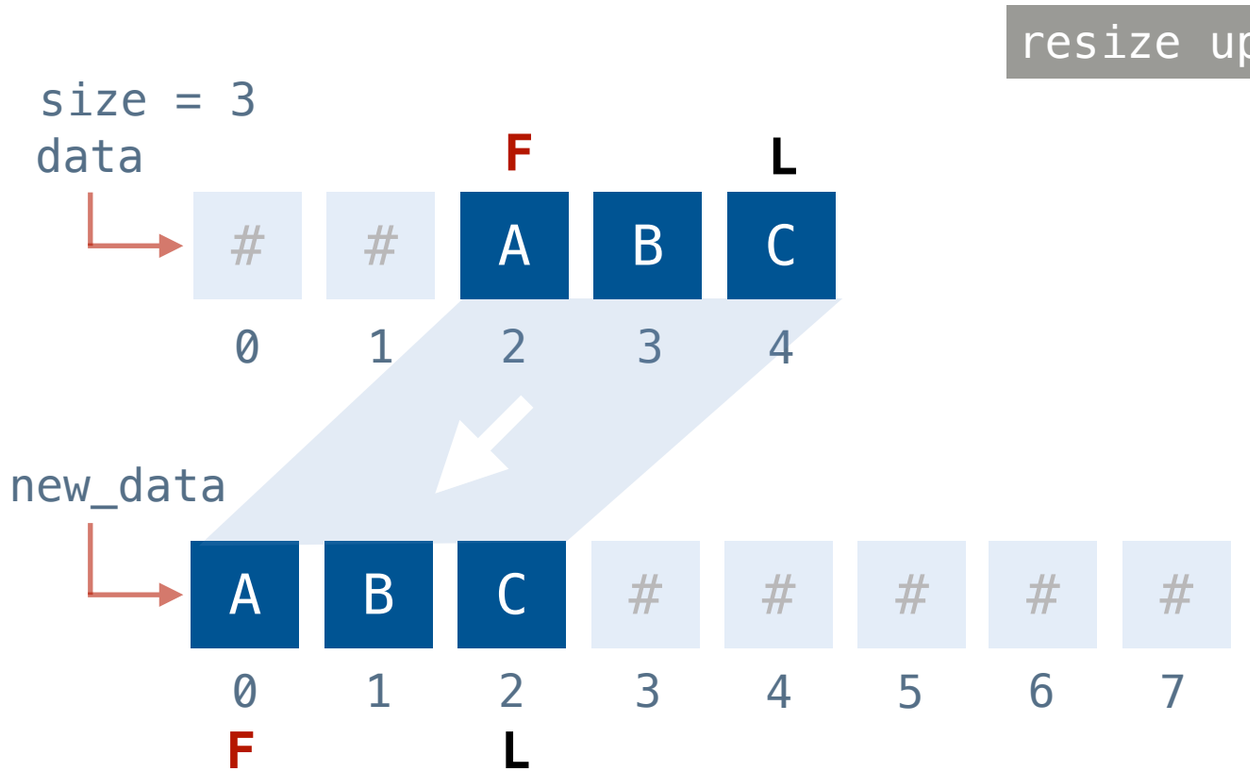


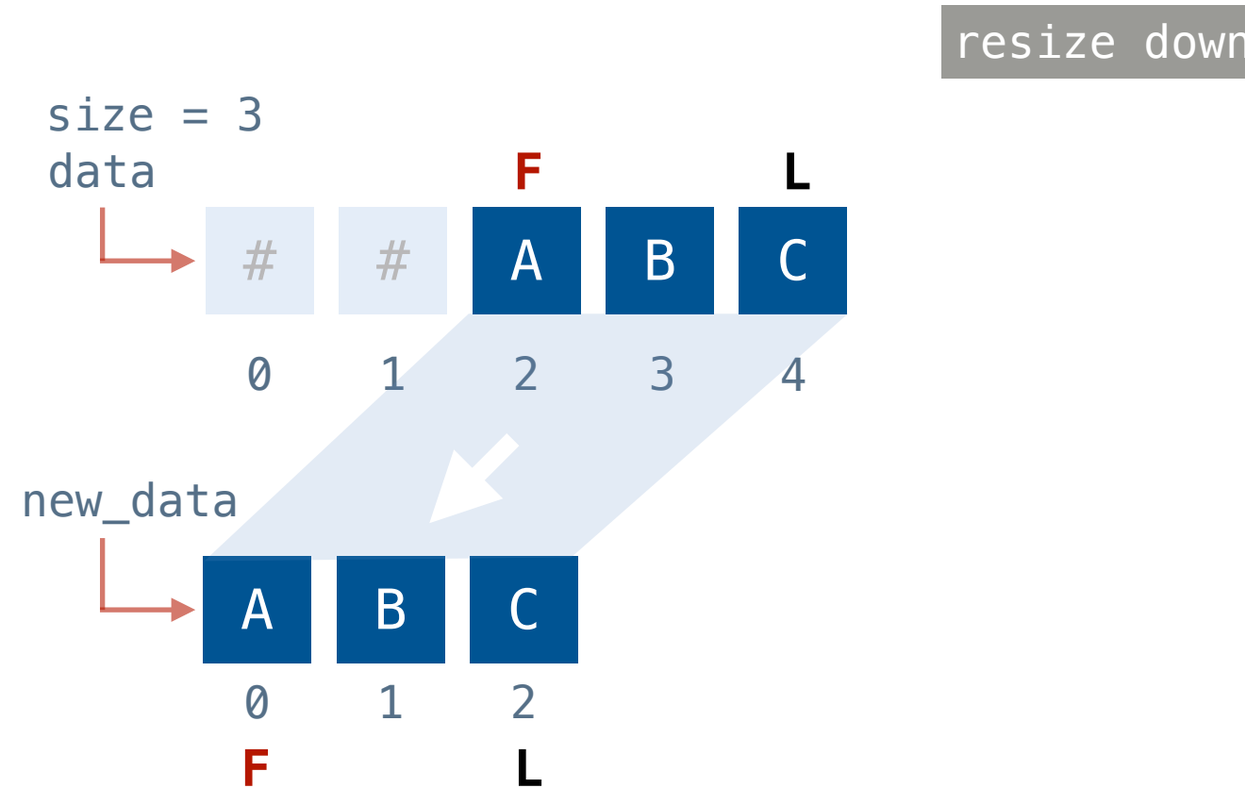
Resizing The Queue (resize and reorder the elements)

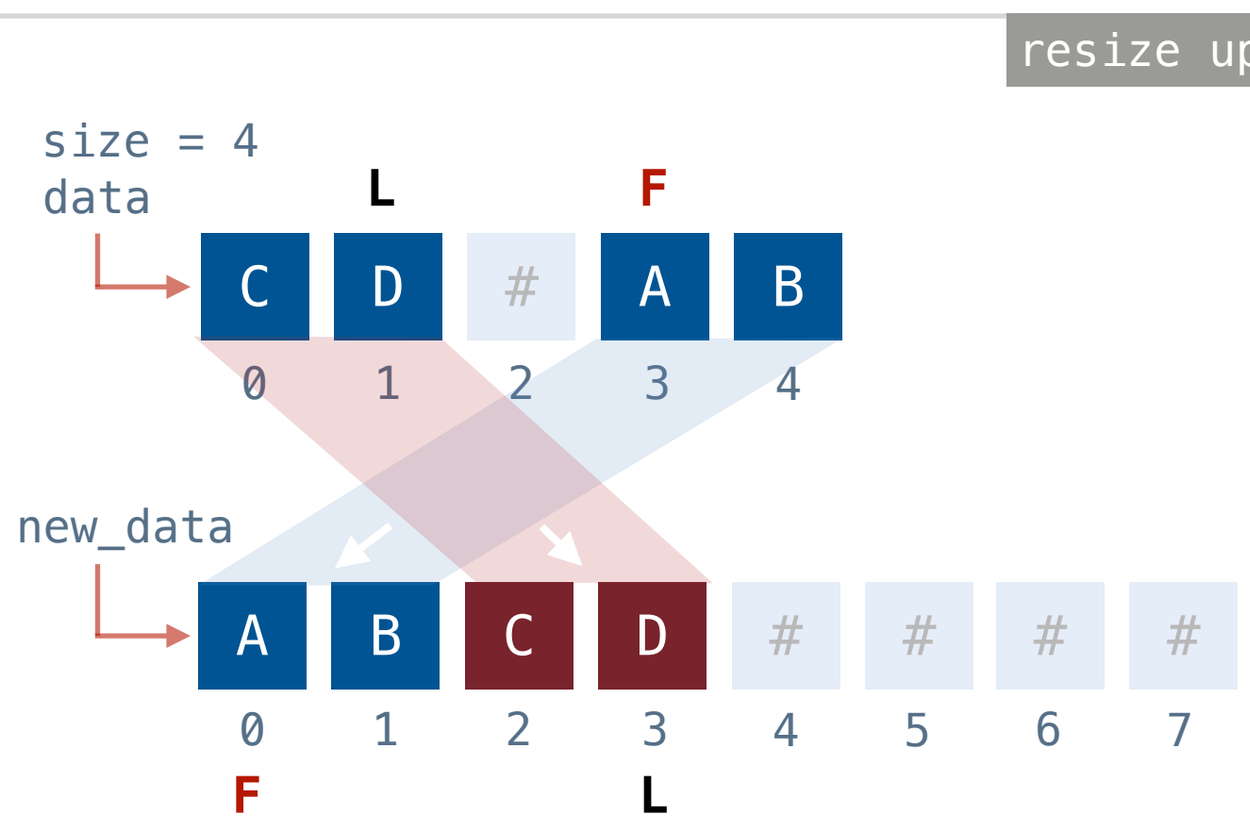




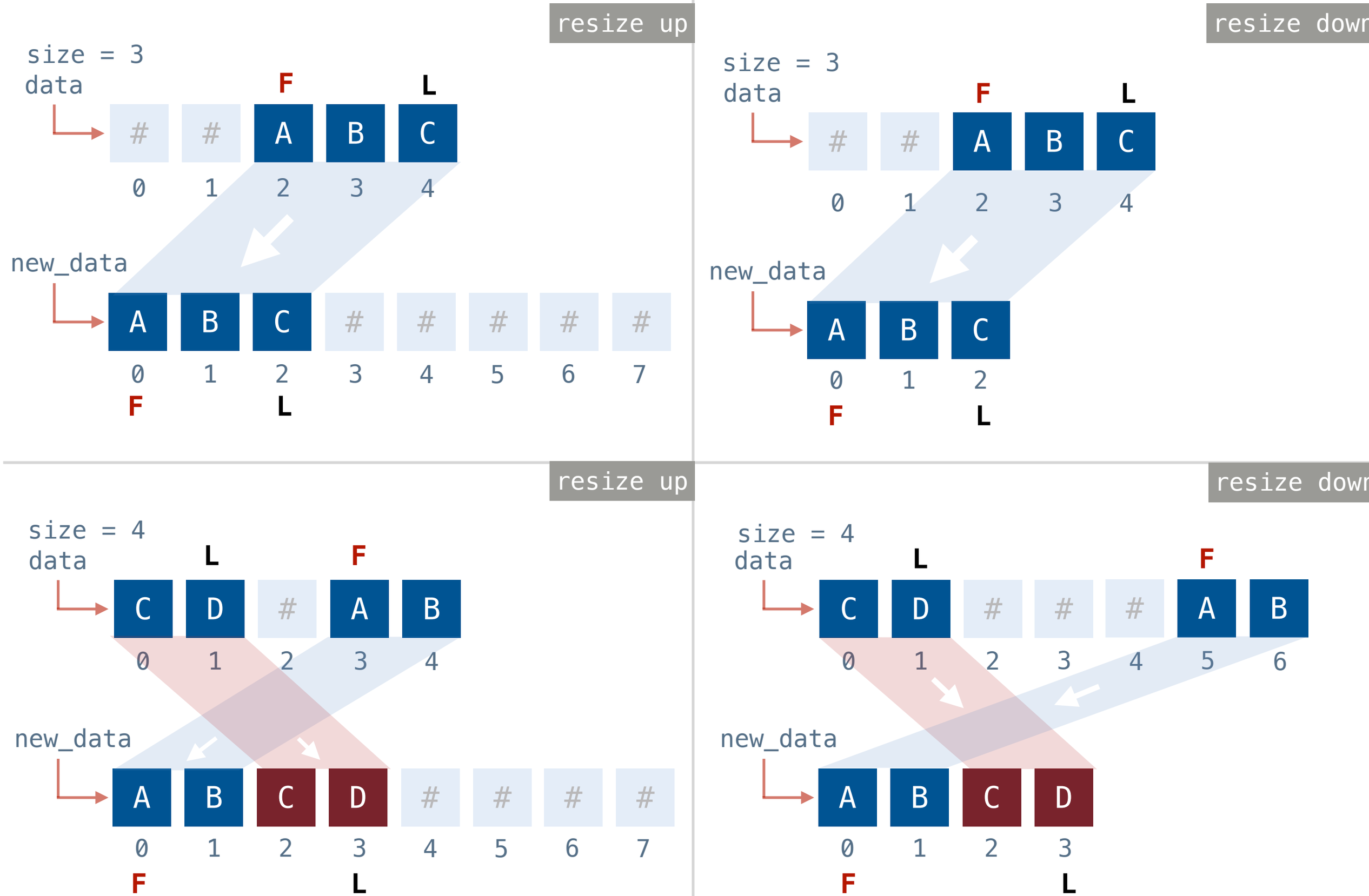
Resizing The Queue (resize and reorder the elements)







Resizing The Queue (resize and reorder the elements)



https://cdn.iconscout.com/icon/premium/png-256-thumb/ticket-queue-1560378-1321382.png
https://chainstrading.net/img/p/1/2/4/9/1249-thickbox_default.jpg